



UNIVERSITÀ DEGLI STUDI DI BERGAMO

Corso di Progettazione, Algoritmi e Computabilità

Documentazione Progetto

Gestionale Feste di Paese : "QuickByte"

Maccari Luca	Matr. 1081951
Masinari Gabriele	Matr. 1079692
Tomasoni Francesco	Matr. 1080980

Anno Accademico 2025/2026

Indice

1	ITERAZIONE 0	1
1.1	Introduzione	1
1.2	Attori del sistema	1
1.3	Use Case Stories	2
1.3.1	Use Case Stories: Cassiere	2
1.3.2	Use Case Stories: Admin	3
1.3.3	Use Case Stories: Sistema di Smistamento	4
1.4	Use Case Diagram	4
1.5	Fully Dressed Descriptions	6
1.6	Requisiti Funzionali	18
1.7	Requisiti Non Funzionali	19
1.8	Topologia del sistema	19
1.9	Stile architetturale	21
1.10	Toolchain e tecnologie utilizzate	22
2	ITERAZIONE 1	23
2.1	Casi d'uso implementati	23
2.2	Diagrammi UML	24
2.2.1	UML Component Diagram	24
2.2.2	UML Class Diagram Interface	25
2.2.3	UML Class Diagram per tipo di dato	26
2.2.4	UML Deployment Diagram	26
2.3	Test	28
2.3.1	Analisi statica del codice	28
2.3.2	Test automatizzati	29
2.3.3	API esposte	30
3	ITERAZIONE 2	36
3.1	Casi d'uso implementati	36
3.2	Diagrammi UML	36
3.2.1	UML Component Diagram	36
3.2.2	UML Class Diagram Interface	37
3.2.3	UML Class Diagram per tipo di dato	39
3.2.4	UML Deployment Diagram	41
3.3	Algoritmo: Smistatore Intelligente Multi-Reparto	42
3.4	Test	49
3.4.1	Analisi statica del codice	49
3.4.2	Test automatizzati	50
3.4.3	API esposte	52
4	ITERAZIONE 3	55
4.1	Casi d'uso implementati	55
4.2	Realizzazione dell'autenticazione Admin (UC8)	55
4.3	Diagrammi UML	56
4.4	Algoritmo: Predittore Dinamico di Riordino Scorte	59

4.5	Test	66
4.5.1	Analisi statica del codice e refactoring	66
4.5.2	Test automatizzati	69
4.5.3	Analisi di copertura del backend	71
4.5.4	API esposte	73

Elenco delle figure

1	Use Case Diagram del sistema	5
2	Topology Diagram	20
3	Stile architetturale 3-tier	21
4	UML Component Diagram	24
5	UML Class Diagram per interfacce	25
6	UML Class Diagram per tipo di dato	26
7	UML Deployment Diagram	27
8	Complessità ciclomatica delle funzioni più impegnative del backend di base. Solo parsaFlusso (parser ESC/POS) e il bootstrap del server (avviaServer) superano la soglia di 15.	28
9	Accoppiamento: numero di dipendenze interne (fan-out) dei moduli più connessi. Complessivamente 7 moduli su 10 dipendono da db.js ; nessuna dipendenza circolare.	29
10	Dimensione dei moduli più grandi del backend di base, in righe di codice (SLOC).	29
11	Esecuzione della suite di test del sistema base con Vitest: 47/47 test superati sui moduli Ordini, Menu, Scorte e Stampanti.	29
12	Postman — POST /api/ordini	31
13	Postman — POST /api/ordini/:id/conferma	32
14	Postman — POST /api/menu	33
15	Postman — PUT /api/menu/:id	33
16	Postman — PUT /api/scorte/:voce_id	34
17	Postman — POST /api/scorte/:voce_id/rifornimento	34
18	Postman — POST /api/stampanti	35
19	Postman — PUT /api/stampanti/:id	35
20	Diagramma dei Componenti UML Iterazione 2	37
21	Diagramma UML delle Classi per interfacce Iterazione 2	38
22	Diagramma UML di Classe per tipo di dato Iterazione 2	40
23	Diagramma UML di Deployment Iterazione 2	41
24	Pseudocodice funzione principale ROUTEORDER	46
25	Pseudocodice funzioni ausiliarie	47
26	Diagramma di flusso Smistatore Intelligente Multi-Reparto	48
27	Complessità ciclomatica delle funzioni principali del backend. Le tre funzioni in basso (routeOrder , batchPerReparto , rigaToTasks) sono introdotte dallo Smistatore e restano tutte sotto la soglia di 15; le altre, legate all'infrastruttura di base, sono invariate.	49
28	Metriche strutturali prima e dopo l'aggiunta del solo Smistatore: il modulo aggiunge due moduli al grafo (route e service), di cui uno (la route) accoppiato al database.	50
29	Dimensione del backend in righe di codice (SLOC) prima e dopo l'aggiunta del solo Smistatore.	50
30	Esecuzione della suite unitaria dello Smistatore con Vitest: 26/26 test unitari superati	51
31	Postman POST /api/smistatore/completa	52

32	Postman PUT /api/smistatore/stampante/:reparto	53
33	Postman PUT /api/smistatore/fallback/:reparto	53
34	UML Component Diagram aggiornato con il componente GestioneAutenticazione e l'interfaccia AutenticazioneMgmt — Iterazione 3	57
35	UML Class Diagram per interfacce aggiornato con AutenticazioneMgmt — Iterazione 3	58
36	Diagramma di flusso Predittore Dinamico di Riordino Scorte	65
37	Complessità ciclomatica delle funzioni più critiche	67
38	Accoppiamento prima e dopo.	67
39	Coesione vista come decomposizione dei moduli: i due “God Component” del frontend e il modulo più grande del progetto vengono divisi in unità a responsabilità singola.	67
40	Esecuzione della suite unitaria del Predittore con Vitest: 33/33 test unitari superati.	69
41	Esecuzione della suite di autenticazione Admin con Vitest.	71
42	Esecuzione completa della suite Vitest con Docker attivo e report di co- pertura: 71/71 test superati (4 file, nessuno saltato), copertura globale al 94,4% di branch (100% su statement, funzioni e righe).	72
43	Postman — PUT /api/predittore/configurazione/:chiave	74
44	Postman — PUT /api/predittore/voce/:id	74
45	Postman — POST /api/menu/aggregata	76
46	Postman — PUT /api/menu/settori/rinomina	76
47	Postman — PUT /api/menu/:id/composizione	77

1 ITERAZIONE 0

1.1 Introduzione

Le feste di paese e le sagre locali sono eventi di grande aggregazione, ma la loro gestione logistica è spesso ostacolata da processi manuali inefficienti: code alle casse, errori nella lettura delle comande cartacee, ritardi in preparazione e disallineamento tra ordini e scorte reali in cucina. Sono problemi ricorrenti che penalizzano sia il lavoro dei volontari sia l'esperienza dei partecipanti.

Questo progetto risolve tali criticità con un sistema digitale e centralizzato per la gestione di comande, cassa e smistamento degli ordini, pensato per ottimizzare l'intero flusso di lavoro, ridurre il margine di errore umano e velocizzare il servizio. La piattaforma offre un front-end rapido e intuitivo per i cassieri e un back-office riservato agli amministratori per la configurazione di menù e regole di business.

Il sistema copre tre ambiti principali:

- **Gestione della cassa e composizione dell'ordine:** interfaccia touch-friendly per la selezione rapida dei piatti, calcolo automatico del totale, gestione flessibile di sconti, calcolo del resto e opzioni dedicate per l'asporto.
- **Gestione e smistamento delle stampe:** invio automatizzato e smistamento intelligente delle comande ai vari reparti di preparazione (es. cucina, pizzeria, bar, griglia) in base a regole di stampa prestabilite, eliminando la necessità di fattorini fisici tra la cassa e i reparti.
- **Gestione dinamica delle scorte:** monitoraggio in tempo reale delle quantità limitate, sia per prodotti singoli che per ingredienti condivisi tra più piatti, con alert visivi automatici per prevenire la vendita di pietanze esaurite.

1.2 Attori del sistema

Gli attori coinvolti nel sistema Gestionale Feste sono:

Attore	Descrizione
Cassiere	Operatore principale della cassa. Gestisce ordini, pagamenti, note e stampe durante la festa.
Amministratore (Admin)	Configura il sistema prima della festa: menu, stampanti, scorte, allergeni. Attore della fase di setup. Nei casi d'uso e nelle iterazioni successive è indicato con il nome breve <i>Admin</i> .
Sistema di Smistamento	Attore secondario. Rappresenta il sottosistema responsabile dello smistamento e dell'invio dei biglietti alle stampanti distribuite. È modellato come attore secondario per enfatizzare la natura distribuita della stampa su più postazioni fisiche, con responsabilità distinte rispetto alla gestione dell'ordine.

Tabella 1: Attori del sistema

1.3 Use Case Stories

1.3.1 Use Case Stories: Cassiere

CONSULTARE ALLERGENI

- Come cassiere, voglio poter consultare la lista degli allergeni di ogni pietanza per informare i clienti con intolleranze alimentari.
- Come cassiere, voglio che la finestra degli allergeni sia accessibile in qualsiasi momento durante la composizione dell'ordine.
- Come cassiere, voglio poter stampare la scheda allergeni di un prodotto o l'elenco completo se richiesto dal cliente.

AGGIUNGERE PIETANZE ALL'ORDINE

- Come cassiere, voglio selezionare le pietanze cliccando sui tasti nella parte sinistra dello schermo per aggiungere velocemente gli articoli al riepilogo.
- Come cassiere, voglio che il riepilogo si aggiorni in tempo reale con quantità e prezzi complessivi ad ogni selezione.
- Come cassiere, voglio che i tasti delle pietanze esaurite siano visivamente distinti (grigio scuro, non cliccabili) per evitare errori.
- Come cassiere, voglio ricevere un avviso visivo (lampeggio giallo/rosso) quando le scorte di un prodotto stanno per esaurirsi.
- Come cassiere, voglio ricevere un alert non bloccante se ordino una pietanza il cui ingrediente condiviso (es. polenta) è esaurito.

MODIFICARE PIETANZA ORDINATA

- Come cassiere, voglio poter accedere alla schermata di modifica cliccando su una pietanza nel riepilogo per correggere quantità e note.
- Come cassiere, voglio avere tasti di decremento rapido (-1, -2, ... -N) per ridurre velocemente le porzioni di una pietanza.
- Come cassiere, voglio poter aggiungere una nota libera per ogni singola porzione di una pietanza per personalizzare l'ordine.
- Come cassiere, voglio poter selezionare note preimpostate da un menù rapido per velocizzare le variazioni più comuni.
- Come cassiere, voglio poter eliminare completamente una pietanza dall'ordine con un singolo tasto.

GESTIONE ORDINE

- Come cassiere, voglio poter azzerare l'intero ordine con un tasto dedicato per ricominciare da zero in caso di errore.
- Come cassiere, voglio poter inserire la cifra pagata dal cliente per ottenere automaticamente il calcolo del resto.
- Come cassiere, voglio poter applicare una scontistica all'ordine prima della conferma.
- Come cassiere, voglio poter taggare un ordine come 'asporto' in qualsiasi momento con un tasto che cambia colore per indicarne l'attivazione.

CONFERMARE ORDINE

- Come cassiere, voglio confermare l'ordine con un singolo tasto per avviare la stampa sui settori competenti.
- Come cassiere, voglio ricevere un alert non bloccante prima della stampa se l'ordine asporto contiene pietanze non asportabili.
- Come cassiere, voglio che dopo la conferma il sistema torni automaticamente alla schermata principale per ricevere il prossimo ordine.

1.3.2 Use Case Stories: Admin

AUTENTICARSI

- Come admin, voglio accedere all'area di amministrazione tramite una password riservata, così che solo il personale autorizzato possa configurare il sistema.
- Come admin, voglio che l'accesso richieda la sola password, senza nome utente, per essere rapido durante l'allestimento della festa.
- Come admin, voglio poter cambiare la password di amministrazione dopo l'accesso.
- Come admin, voglio che la sessione di accesso abbia una scadenza, così che una postazione lasciata incustodita non resti sbloccata a tempo indeterminato.

CONFIGURARE MENÙ

- Come admin, voglio poter aggiungere, modificare o rimuovere pietanze/bevande dal menù prima della festa.
- Come admin, voglio poter assegnare ogni pietanza a una colonna di visualizzazione, un colore e un ordine di posizione sullo schermo.
- Come admin, voglio poter nascondere temporaneamente una pietanza dallo schermo senza eliminarla dal database.
- Come admin, voglio poter configurare per ogni pietanza una o più stampanti di destinazione per il settore di stampa.

- Come admin, voglio poter definire note preimpostate per ogni pietanza, con eventuale costo aggiuntivo associato.
- Come admin, voglio poter marcare ogni pietanza come 'da asporto' o 'non da asporto'.

IMPOSTARE SCORTE

- Come admin, voglio poter impostare le quantità iniziali delle pietanze a disponibilità limitata prima dell'inizio della festa.
- Come admin, voglio poter configurare le soglie di allerta (giallo e rosso) per il lampeggio dei tasti a scorte basse.
- Come admin, voglio poter creare contatori aggregati per ingredienti condivisi tra più pietanze (es. polenta) e associarvi i prodotti interessati.
- Come admin, voglio poter creare prodotti 'fittizi' da usare esclusivamente per il monitoraggio dei contatori aggregati.

COMPILARE ALLERGENI

- Come admin, voglio poter associare gli allergeni a ogni pietanza del menù prima della festa.
- Come admin, voglio poter stampare la lista completa di tutte le pietanze con i relativi allergeni.
- Come admin, voglio poter stampare la scheda allergeni di un singolo prodotto.

1.3.3 Use Case Stories: Sistema di Smistamento

STAMPA E AGGIORNAMENTO SCORTE

- Come sistema di smistamento, devo ricevere i biglietti degli ordini confermati e indirizzarli alle stampanti dei settori corretti.
- Come sistema di smistamento, devo notificare al sistema principale eventuali errori di stampa.
- Come sistema di smistamento, devo collaborare all'aggiornamento delle scorte al momento della conferma di ogni ordine.

1.4 Use Case Diagram

Il diagramma dei casi d'uso (Use Case Diagram) del sistema Gestionale Feste è riportato di seguito. Il sistema include due attori primari (Cassiere e Admin) e un attore secondario (Sistema di Smistamento).

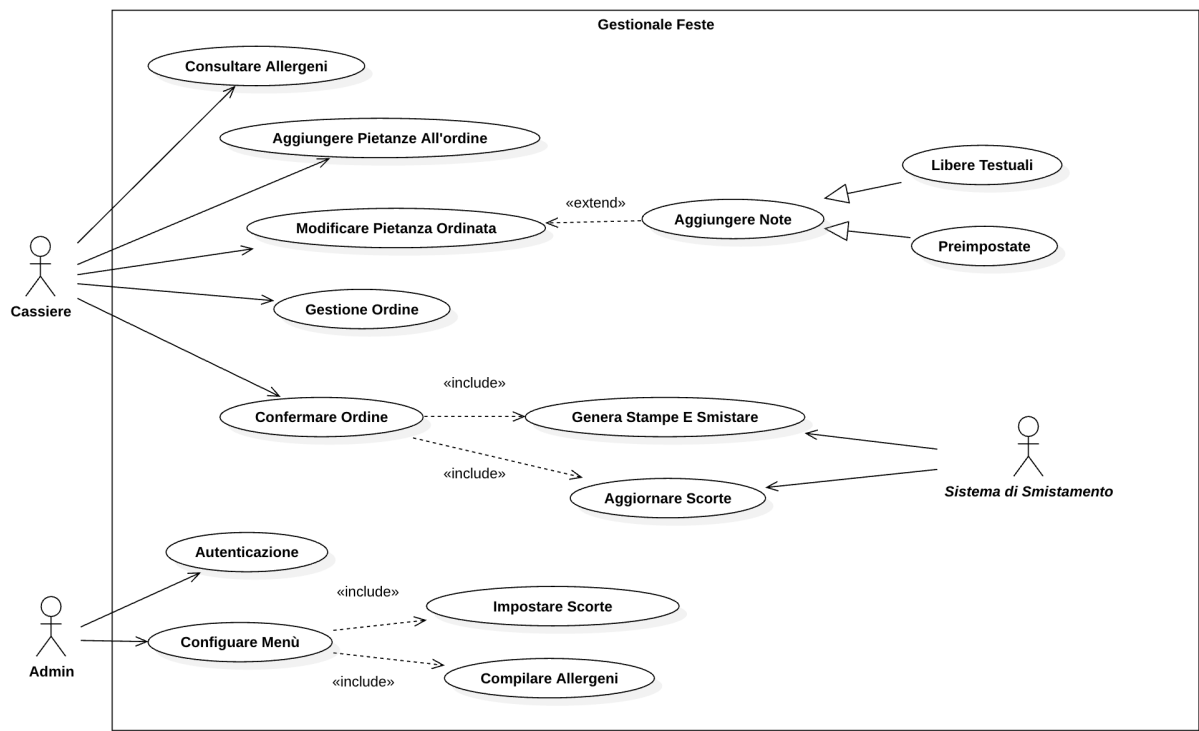


Figura 1: Use Case Diagram del sistema

1.5 Fully Dressed Descriptions

UC1 - Consultare Allergeni

Use Case	Consultare Allergeni
Summary	Il cassiere consulta la lista degli allergeni associati alle pietanze presenti nel menù.
Actor	Cassiere
Precondizione	Il cassiere ha avviato il sistema. Il menù e gli allergeni sono stati preventivamente configurati dall'Admin.
Postcondizione	Il cassiere ha visualizzato gli allergeni del prodotto selezionato.
Base Sequence	1. Il cassiere preme il tasto 'Allergeni' nella parte destra dello schermo. 2. Il sistema apre una finestra con l'elenco dei prodotti divisi per settore di visualizzazione. 3. Il cassiere seleziona una pietanza dalla lista. 4. Il sistema mostra l'elenco degli allergeni contenuti nella pietanza selezionata. 5. Il cassiere chiude la finestra e torna alla schermata principale.
Branch Sequence	
Exception Sequence	E1: Nessun allergene associato alla pietanza. 4. Il sistema mostra il messaggio 'Nessun allergene registrato per questo prodotto'. 5. Il cassiere chiude la finestra.
Sub Use Case	Configurare Menù (compilazione allergeni da parte dell'Admin)
Note	La finestra degli allergeni è accessibile in qualsiasi momento durante la composizione dell'ordine.

UC2 - Aggiungere Pietanze All'ordine

Use Case	Aggiungere Pietanze All'ordine
Summary	Il cassiere seleziona le pietanze ordinate dal cliente aggiungendole al riepilogo dell'ordine.
Actor	Cassiere
Precondizione	Il sistema è avviato e il menù è configurato. La schermata principale è visibile con le colonne pietanze sulla sinistra e il riepilogo sulla destra.
Postcondizione	Le pietanze selezionate appaiono nel riepilogo a destra con quantità e prezzo aggiornati in tempo reale.
Base Sequence	1. Il cassiere identifica la pietanza richiesta dal cliente nella colonna corrispondente (es. Primi, Secondi, Bar). 2. Il cassiere clicca sul tasto della pietanza desiderata. 3. Il sistema aggiunge la pietanza al riepilogo a destra oppure ne incrementa la quantità se già presente. 4. Il sistema aggiorna il totale dell'ordine in tempo reale. 5. Il cassiere ripete i passi 1-4 per ogni ulteriore pietanza ordinata dal cliente.
Branch Sequence	B1: La pietanza è a quantità limitata in esaurimento. 2. Il tasto della pietanza lampeggia di giallo (scorta bassa) o di rosso (scorta critica). 3. Il cassiere può comunque selezionare la pietanza. B2: La pietanza è esaurita (scorta = 0). 2. Il tasto appare grigio scuro e non è selezionabile. 3. Il cassiere informa il cliente della non disponibilità.
Exception Sequence	E1: Il contatore aggregato (es. polenta) è esaurito. 3. Il sistema mostra un alert non bloccante segnalando l'esaurimento dell'ingrediente condiviso. 4. Il cassiere può proseguire o rimuovere la pietanza.
Sub Use Case	—
Note	Ogni clic sul tasto di una pietanza aggiunge una unità. Se la pietanza ha un flag 'non visualizzare su schermo', non compare tra i tasti.

UC3 - Modificare Pietanza Ordinata

Use Case	Modificare Pietanza Ordinata
Summary	Il cassiere modifica una pietanza già inserita nell'ordine, potendo cambiare la quantità e/o aggiungere note.
Actor	Cassiere
Precondizione	Almeno una pietanza è stata aggiunta al riepilogo dell'ordine.
Postcondizione	La pietanza risulta aggiornata nel riepilogo con la nuova quantità e/o le note indicate. Il totale è ricalcolato.
Base Sequence	1. Il cassiere clicca sulla pietanza nel riepilogo a destra dello schermo. 2. Il sistema apre la schermata di modifica, mostrando: nome piatto, prezzo, quantità ordinata. 3. Il cassiere visualizza le opzioni disponibili: modifica quantità, note preimpostate, note libere. 4. Il cassiere effettua le modifiche desiderate. 5. Il cassiere conferma le modifiche. 6. Il sistema aggiorna il riepilogo e il totale dell'ordine.
Branch Sequence	B1: Il cassiere sceglie di eliminare completamente la pietanza. 4. Il cassiere preme il tasto 'Elimina pietanza'. 5. Il sistema rimuove la pietanza dal riepilogo e aggiorna il totale.
Exception Sequence	
Sub Use Case	UC3b - Aggiungere Note
Note	La schermata di modifica mostra tanti tasti di decremento quante sono le porzioni ordinate (es. 3 porzioni → tasti -1, -2, -3) più il tasto +1.

UC3b - Aggiungere Note

Use Case	Aggiungere Note
Summary	Il cassiere aggiunge una nota testuale a una o più porzioni di una pietanza nell'ordine, utilizzando testo libero o note preimpostate.
Actor	Cassiere
Precondizione	La schermata di modifica della pietanza è aperta (UC3).
Postcondizione	Le note sono associate alla pietanza e verranno stampate sullo scontrino.
Base Sequence	1. Il cassiere visualizza i campi nota (uno per ogni porzione ordinata). 2. Il cassiere seleziona il campo nota della porzione di interesse. 3. Il cassiere sceglie se inserire una nota preimpostata o una nota libera. 4. Il sistema registra la nota associata alla specifica porzione. 5. Il cassiere può ripetere per le altre porzioni della stessa pietanza.
Branch Sequence	B1: Nota preimpostata con costo aggiuntivo. 4. Il sistema aggiunge il costo della nota al prezzo della pietanza e aggiorna il totale dell'ordine.
Exception Sequence	
Sub Use Case	—
Note	Questo caso d'uso è raggiunto tramite relazione «extend» da UC3. Il numero di campi nota corrisponde alla quantità di porzioni ordinate.

UC4 - Gestione Ordine

Use Case	Gestione Ordine
Summary	Il cassiere gestisce le opzioni generali dell'ordine corrente: applicazione di sconti, marcatura come asporto e gestione del pagamento.
Actor	Cassiere
Precondizione	Il sistema è avviato. Almeno una pietanza è stata aggiunta all'ordine oppure l'ordine è in fase di composizione.
Postcondizione	Le opzioni selezionate (sconto e/o asporto) sono registrate nell'ordine e verranno applicate alla conferma e alla stampa.
Base Sequence	1. Il cassiere compone l'ordine aggiungendo pietanze. 2. Il cassiere valuta se applicare opzioni aggiuntive (sconto, asporto). 3. Il cassiere utilizza le funzioni di estensione disponibili. 4. Il sistema registra le opzioni nell'ordine corrente. 5. Il cassiere procede alla conferma dell'ordine (UC5). 6. (Opzionale) Il cassiere inserisce l'importo pagato dal cliente; il sistema calcola e visualizza automaticamente il resto.
Branch Sequence	- B1: Il cassiere decide di azzerare completamente l'ordine. 2. Il cassiere preme il tasto 'Azzerà ordine'. 3. Il sistema cancella tutte le pietanze dal riepilogo e azzerà il totale.
Exception Sequence	
Sub Use Case	—
Note	La gestione ordine avviene nella parte destra dello schermo.

UC5 - Confermare Ordine

Use Case	Confermare Ordine
Summary	Il cassiere conferma l'ordine, che viene inviato in stampa alle stampanti dei settori competenti e aggiorna le scorte.
Actor	Cassiere
Precondizione	L'ordine è composto (almeno una pietanza presente). Le stampanti associate ai settori di stampa sono configurate e attive.
Postcondizione	L'ordine è confermato e inviato in stampa. Le scorte vengono aggiornate. Il sistema è pronto per un nuovo ordine.
Base Sequence	1. Il cassiere preme il tasto 'Conferma Ordine'. 2. Il sistema verifica l'ordine (pietanze, note, sconto, flag asportato). 3. Il sistema avvia il processo di stampa (UC5a - Genera Stampe e Smistare) e l'aggiornamento scorte (UC5b - Aggiornare Scorte). 4. Il sistema azzera il riepilogo e lo schermo torna alla schermata principale pronto per un nuovo ordine.
Branch Sequence	- B1: Ordine asportato con pietanze non asportabili. 2. Il sistema mostra un alert non bloccante. 3. Il cassiere sceglie di proseguire: le pietanze non asportabili vengono poste in coda al biglietto.
Exception Sequence	- E1: Una o più stampanti non sono raggiungibili. 3. Il sistema notifica l'errore di stampa. 4. Il cassiere verifica lo stato delle stampanti e ritenta.
Sub Use Case	UC5a - Genera Stampe E Smistare, UC5b - Aggiornare Scorte
Note	UC5a e UC5b sono inclusi tramite relazione «include» e vengono sempre eseguiti alla conferma dell'ordine.

UC5a - Genera Stampe E Smistare

Use Case	Genera Stampe E Smistare
Summary	Il sistema genera i biglietti di stampa per ogni settore (cucina, bar, pizzeria, ecc.) e li invia alle stampanti associate, coordinandosi con il Sistema di Smistamento.
Actor	Cassiere, Sistema di Smistamento
Precondizione	L'ordine è stato confermato (UC5). Le stampanti associate ai settori di stampa sono configurate.
Postcondizione	Ogni pietanza è stampata sul biglietto del settore di competenza e inviata alla/e stampante/e corretta/e. Eventuale copia cliente emessa sulla stampante cassa.
Base Sequence	1. Il sistema raggruppa le pietanze per settore di stampa (cucina, bar, pizzeria, ecc.). 2. Per ogni settore, il sistema genera un biglietto distinto. 3. Il sistema invia ogni biglietto alle stampanti associate al relativo settore. 4. Se nell'ordine sono presenti pietanze con modalità 'Stampa singola quantità singola', vengono stampati tanti biglietti singoli quante le porzioni ordinate. 5. Se nell'ordine sono presenti pietanze con modalità 'Stampa singola quantità multipla', viene stampato un biglietto per quella pietanza con la quantità totale. 6. Per i prodotti con flag 'Copia scontrino cliente', viene emessa una copia cliente sulla stampante cassa. 7. Se l'ordine è asporto, ogni biglietto riporta il titolo 'ASPOR-TO' in alto a caratteri cubitali.
Branch Sequence	
Exception Sequence	E1: Stampante non raggiungibile. 3. Il Sistema di Smistamento notifica l'errore al sistema principale. 4. Il sistema informa il cassiere della mancata stampa su quel settore.
Sub Use Case	UC5 - Confermare Ordine
Note	Raggiunto tramite relazione «include» da UC5. Il Sistema di Smistamento è un attore secondario che partecipa attivamente a questo caso d'uso.

UC5b - Aggiornare Scorte

Use Case	Aggiornare Scorte
Summary	Il sistema aggiorna automaticamente le scorte residue di ogni prodotto ordinato al momento della conferma dell'ordine.
Actor	Cassiere, Sistema di Smistamento
Precondizione	L'ordine è stato confermato (UC5). La gestione delle scorte è attiva per almeno un prodotto.
Postcondizione	Le scorte dei prodotti ordinati sono decrementate della quantità ordinata. I tasti delle pietanze aggiornano il proprio stato visivo (normale, lampeggio giallo, lampeggio rosso, grigio).
Base Sequence	1. Il sistema identifica le pietanze nell'ordine che hanno la gestione scorte attiva. 2. Per ogni pietanza, il sistema decrementa la scorta residua della quantità ordinata. 3. Per i contatori aggregati (es. polenta), il sistema decrementa il contatore condiviso. 4. Il sistema aggiorna lo stato visivo dei tasti: lampeggio giallo se scorte $\leq$ soglia X, lampeggio rosso se scorte $\leq$ soglia Y, grigio scuro se scorte = 0. 5. Se la scorta di un contatore aggregato raggiunge 0, i tasti delle pietanze associate diventano grigio chiaro (selezionabili con alert).
Branch Sequence	
Exception Sequence	- E1: La scorta risulta inferiore alla quantità ordinata. 2. Il sistema registra comunque l'ordine ma segnala al cassiere l'anomalia.
Sub Use Case	UC5 - Confermare Ordine
Note	Raggiunto tramite relazione «include» da UC5. Le soglie X e Y sono parametri configurabili dall'Admin in una tabella parametrica, con $X > Y > 0$.

UC6 - Configurare Menù

Use Case	Configurare Menù
Summary	L'Admin configura il menù del sistema impostando le pietanze/bevande disponibili, le relative colonne di visualizzazione, i colori, l'ordinamento, gli allergeni, le scorte iniziali e le stampanti di reparto.
Actor	Admin
Precondizione	L'Admin ha accesso al pannello di configurazione del sistema.
Postcondizione	Il menù è configurato e disponibile per il Cassiere. Le pietanze compaiono nelle colonne corrette con i parametri impostati. Le stampanti sono associate ai rispettivi settori di stampa.
Base Sequence	1. L'Admin accede al pannello di configurazione del menù. 2. Il sistema mostra la lista delle pietanze/bevande esistenti. 3. Per ogni pietanza, l'Admin può aggiungere, modificare o rimuovere i seguenti parametri: Codice piatto, Descrizione, Prezzo, Settore di visualizzazione (colonna), Ordine sullo schermo, Colore del tasto, Visualizzare su schermo (booleano), Settore di stampa, Flag asporto/non asporto. 4. L'Admin configura anche le note preimpostate per ogni pietanza con relativo eventuale costo aggiuntivo. 5. L'Admin avvia le procedure di inclusione: Impostare Scorte (UC6a) e Compilare Allergeni (UC6b). 6. L'Admin salva la configurazione. 7. Il sistema rende disponibili le modifiche sulla schermata del Cassiere.
Branch Sequence	
Exception Sequence	E1: Codice piatto duplicato. 6. Il sistema segnala il conflitto e richiede la correzione prima di salvare.
Sub Use Case	UC6a - Impostare Scorte, UC6b - Compilare Allergeni
Note	Il menù può avere fino a 6 colonne di visualizzazione. L'altezza delle celle si adatta automaticamente alla colonna. L'ordinamento segue il campo 'Ordine sullo schermo' crescente; a parità di numero, ordine alfabetico.

UC6a - Impostare Scorte

Use Case	Impostare Scorte
Summary	L'Admin imposta la quantità residua iniziale per i prodotti disponibili in quantità limitata e configura i contatori aggregati per ingredienti condivisi tra più pietanze.
Actor	Admin
Precondizione	Il pannello di configurazione menù è aperto (UC6). Il prodotto a scorta limitata è già presente nel menù.
Postcondizione	Le scorte iniziali sono impostate. I contatori aggregati sono configurati con i relativi prodotti associati.
Base Sequence	1. L'Admin preme il tasto di gestione scorte nel pannello di configurazione. 2. Il sistema apre una tabella con tutti i prodotti. 3. L'Admin imposta la quantità residua per ogni prodotto a scorta limitata. 4. L'Admin configura i valori soglia X (lampeggio giallo) e Y (lampeggio rosso) nella tabella parametrica. 5. L'Admin crea i contatori aggregati per gli ingredienti condivisi (es. polenta), associandovi i prodotti interessati. 6. L'Admin salva la configurazione delle scorte.
Branch Sequence	B1: Prodotto fittizio (non fisicamente ordinabile ma usato per tracking). 3. L'Admin configura il prodotto come 'fittizio' per il solo scopo di monitoraggio del contatore.
Exception Sequence	E1: Valore soglia non valido ($X \leq Y$ o $Y \leq 0$). 4. Il sistema segnala l'errore e richiede la correzione (vincolo $X > Y > 0$).
Sub Use Case	UC6 - Configurare Menù
Note	Raggiunto tramite relazione «include» da UC6. I contatori aggregati vengono decrementati ogni volta che uno qualsiasi dei prodotti associati viene ordinato.

UC6b - Compilare Allergeni

Use Case	Compilare Allergeni
Summary	L'Admin associa gli allergeni a ogni pietanza del menù, rendendo disponibile la consultazione da parte del Cassiere.
Actor	Admin
Precondizione	Il pannello di configurazione menù è aperto (UC6). Le pietanze sono già presenti nel menù.
Postcondizione	Gli allergeni sono associati alle pietanze. Il Cassiere può consultarli tramite il tasto 'Allergeni'. È possibile stampare la lista completa o quella di un singolo prodotto.
Base Sequence	1. L'Admin accede alla sezione 'Allergeni' nel pannello di configurazione. 2. Il sistema mostra la lista dei prodotti divisi per settore di visualizzazione. 3. L'Admin seleziona un prodotto. 4. Il sistema mostra i campi per l'inserimento degli allergeni. 5. L'Admin inserisce o modifica l'elenco degli allergeni per il prodotto. 6. L'Admin salva le informazioni. 7. L'Admin ripete i passi 3-6 per ogni prodotto che necessita di allergeni. 8. L'Admin può stampare la lista completa di prodotti e allergeni o quella di un singolo prodotto.
Branch Sequence	
Exception Sequence	
Sub Use Case	UC6 - Configurare Menù
Note	Raggiunto tramite relazione «include» da UC6. Questa configurazione avviene tipicamente prima della festa/evento.

UC8 - Autenticazione Admin

Use Case	Autenticazione Admin
Summary	L'Admin accede all'area di amministrazione inserendo la password riservata; il sistema la verifica e abilita le funzioni di backoffice per la durata della sessione.
Actor	Admin
Precondizione	Il sistema è avviato. È stata definita una password di amministrazione (di default 1111 al primo avvio, eventualmente già modificata).
Postcondizione	In caso di successo l'Admin ottiene una sessione valida e può operare sul backoffice. In caso di fallimento l'accesso resta negato e nessuna sessione viene creata.
Base Sequence	1. L'Admin apre l'area di amministrazione. 2. Il sistema, non rilevando una sessione valida, mostra la schermata di accesso con il solo campo password. 3. L'Admin inserisce la password e conferma. 4. Il sistema verifica la password contro quella memorizzata per l'amministratore. 5. La password è corretta: il sistema apre una sessione con scadenza e mostra il pannello di amministrazione. 6. Nelle operazioni successive l'Admin opera entro la sessione autenticata.
Branch Sequence	- B1: Sessione già attiva. 2. Il sistema riconosce una sessione valida e mostra direttamente il pannello, senza richiedere di nuovo la password. - B2: Cambio password. 5. Dall'area amministrativa l'Admin fornisce password attuale e nuova; il sistema verifica la prima e registra la seconda.
Exception Sequence	- E1: Password errata. 4. Il sistema nega l'accesso e non crea alcuna sessione; l'Admin resta sulla schermata di accesso. - E2: Sessione scaduta o assente su una funzione riservata. 6. Il sistema richiede nuovamente l'autenticazione. - E3: Campo password vuoto. 3. Il sistema segnala l'input mancante senza tentare la verifica.
Sub Use Case	Presupposto dai casi d'uso di amministrazione (UC6 - Configurare Menù e correlati), che richiedono l'avvenuta autenticazione.
Note	L'accesso usa una sola password, senza username. È previsto un meccanismo di sessione con scadenza.

1.6 Requisiti Funzionali

- RF-01** **Gestione ordine:** il cassiere può aggiungere pietanze all'ordine cliccando i tasti nella parte sinistra dello schermo. Ogni clic aggiunge o incrementa la pietanza nel riepilogo.
- RF-02** **Modifica quantità e note:** dalla parte destra è possibile modificare le quantità e aggiungere note libere o preimpostate per ogni unità di pietanza ordinata.
- RF-03a** **Calcolo totale e sconti:** il sistema calcola automaticamente il totale dell'ordine ad ogni variazione e applica gli sconti inseriti dal cassiere tramite l'apposita funzione.
- RF-03b** **Gestione pagamento e resto:** il cassiere può inserire l'importo pagato dal cliente; il sistema calcola e visualizza automaticamente il resto dovuto.
- RF-04** **Gestione asporto:** possibilità di taggare l'ordine come asporto con alert per pietanze "non da asporto".
- RF-05** **Gestione stampanti:** le pietanze sono raggruppate per settore di stampa e inviate alle stampanti configurate; gestione di biglietti singoli, multipli e copia cliente.
- RF-06** **Scorte limitate:** gestione visiva delle scorte con soglie di allerta (lampeggio giallo a soglia X, lampeggio rosso a soglia Y). I prodotti singoli con scorta pari a zero diventano grigio scuro e non sono selezionabili (blocco totale). I prodotti legati a un contatore aggregato esaurito diventano grigio chiaro e rimangono selezionabili, ma il sistema mostra un alert non bloccante ad ogni selezione.
- RF-07** **Allergeni:** consultazione e stampa degli allergeni per singola pietanza o per l'intero menu.
- RF-08** **Configurazione menu:** l'amministratore configura pietanze, colonne, ordine, colori, note preimpostate, settori di stampa e flag asporto.
- RF-09** **Configurazione stampanti:** l'amministratore associa una o più stampanti fisiche a ciascun settore di stampa tramite il pannello di configurazione, senza necessità di modificare il codice sorgente.
- RF-10** **Configurazione scorte e contatori:** l'amministratore imposta quantità iniziali, soglie X e Y, e contatori aggregati per ingredienti condivisi.
- RF-11** **Conferma ordine:** il cassiere finalizza l'ordine con un singolo tasto; il sistema verifica le pietanze, avvia la stampa sui settori competenti, aggiorna le scorte e azzera il riepilogo per predisporre al prossimo ordine.
- RF-12** **Autenticazione Admin:** l'accesso all'area di amministrazione è protetto da una password riservata (senza nome utente); la sessione ha una scadenza e la password è modificabile dall'amministratore.

1.7 Requisiti Non Funzionali

Efficienza

Il sistema deve garantire tempi di risposta pressoché istantanei. L'aggiornamento del riepilogo dell'ordine, il ricalcolo dei totali e lo scalo delle scorte limitate devono avvenire in tempo reale per non rallentare le code alla cassa.

Usabilità

Le schermate devono adattarsi alla risoluzione, occupando la parte sinistra per il menu e la destra per il riepilogo. La gestione delle note preimpostate e la codifica a colori dei tasti deve velocizzare le interazioni.

Affidabilità

Il software deve gestire correttamente la coda di stampa comunicando in modo robusto con le stampanti dislocate nei vari reparti, garantendo l'integrità dei dati durante i picchi di affluenza.

Configurabilità

Tutti i parametri chiave (soglie scorte, colori tasti, note preimpostate, settori di stampa, associazioni stampanti) devono essere configurabili senza modificare il codice sorgente.

1.8 Topologia del sistema

La topologia adottata riflette il requisito non funzionale di efficienza e affidabilità: tutte le componenti del sistema operano all'interno di una rete locale (LAN), eliminando la dipendenza da connessioni Internet e garantendo tempi di risposta pressoché istantanei anche durante i picchi di affluenza alla cassa.

I client attivi, rappresentati dalle due postazioni cassa e dal pannello di back-office, comunicano con il server centrale tramite due protocolli applicativi distinti: **HTTP/REST** per le chiamate API (creazione ordini, configurazione menù, gestione scorte) e **WebSocket** per la ricezione in tempo reale degli aggiornamenti sulle scorte, in modo da mantenere l'interfaccia della cassa sempre aggiornata senza necessità di polling.

Il televisore di sala è un client passivo: non effettua mai richieste al server ma si limita a ricevere notifiche in tempo reale sugli ordini pronti tramite **WebSocket**. Quando il personale di cucina segna un ordine come pronto, il server notifica tutti i client connessi e il televisore aggiorna automaticamente la schermata. Non essendo necessaria alcuna interazione con le API, per questo device è sufficiente il solo protocollo **WebSocket**.

Tutti i protocolli applicativi transitano sulla rete locale tramite **IEEE 802.3** (Ethernet) o **Wi-Fi 802.11**.

Il server centrale è l'unico nodo che accede al database relazionale **MySQL/MariaDB**, con cui comunica tramite il protocollo nativo **MySQL Protocol** sulla porta 3306. Le stampanti di reparto (cucina, bar, pizzeria) ricevono i biglietti di stampa direttamente dal server tramite il protocollo **ESC/POS over TCP** sulla porta 9100, standard per le stampanti termiche da ristorazione.

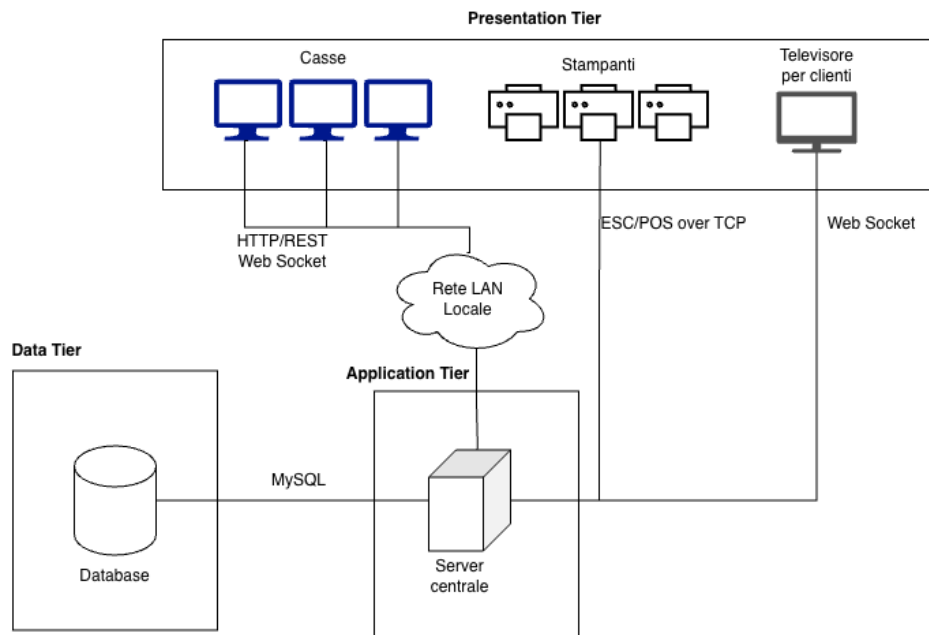


Figura 2: Topology Diagram

1.9 Stile architetturale

Il sistema adotta lo **stile architetturale 3-tier** (Figura 3), che suddivide l'applicazione in tre livelli logicamente e fisicamente separati.

- **Presentation tier:** le postazioni cassa e il client back-office costituiscono il livello di presentazione. Sono applicazioni web eseguite nel browser che si occupano esclusivamente di visualizzare i dati e raccogliere l'input dell'operatore, senza contenere alcuna logica di business.
- **Application tier:** il server centrale ospita tutta la logica applicativa. Riceve le richieste dai client tramite HTTP/WebSocket, le elabora (gestione degli ordini, calcolo dei totali, aggiornamento in tempo reale delle scorte, smistamento delle stampe ai reparti) e restituisce le risposte. È l'unico livello che comunica direttamente con il database e con le stampanti.
- **Data tier:** il database relazionale MySQL/MariaDB gestisce la persistenza di tutti i dati: configurazione del menu, scorte limitate, ordini confermati e impostazioni delle stampanti. Le stampanti distribuite nei reparti appartengono anch'esse a questo livello in quanto rappresentano il punto terminale di output dei dati.

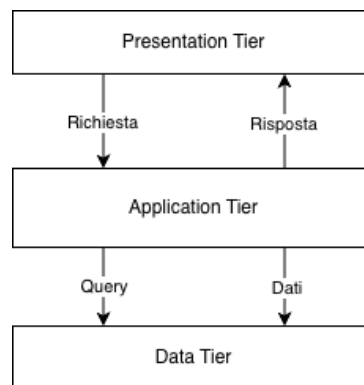


Figura 3: Stile architetturale 3-tier

Questo stile garantisce vantaggi concreti per il progetto:

- **Manutenibilità:** ogni livello può essere modificato indipendentemente dagli altri; si può, ad esempio, aggiornare l'interfaccia della cassa senza toccare la logica applicativa o il database.
- **Scalabilità:** è possibile aggiungere nuove postazioni cassa o nuove stampanti di reparto senza modificare il codice sorgente, semplicemente connettendole alla rete locale e aggiornando la configurazione dal pannello di amministrazione.
- **Centralizzazione:** tutta la logica risiede sul server e le postazioni cassa restano semplici terminali leggeri, raggiungibili da qualsiasi dispositivo dotato di browser senza installare software dedicato.

1.10 Toolchain e tecnologie utilizzate

Tabella 2: Toolchain e tecnologie utilizzate

TOOL	FUNZIONE
Visual Studio Code	Sviluppo del codice sorgente.
Git e GitHub	Piattaforma per il controllo versione.
Docker	Ambiente di sviluppo e database locale (MySQL/MariaDB).
Framework jQuery Ajax	Framework per effettuare richieste HTTP/Rest.
PlantUML / Draw.io	Realizzazione di grafici e diagrammi UML.
Google Meet, Discord	Piattaforma di comunicazione per meeting in remoto.
L^AT_EX	Stesura e formattazione della documentazione di progetto.
ESLint	Analisi statica del codice e calcolo della complessità ciclomatica.
Madge	Analisi statica per mappare l'accoppiamento dei moduli e le dipendenze circolari.
Vitest (con v8)	Esecuzione di test automatizzati (unitari e di integrazione HTTP) e report di copertura.
Postman	Testing manuale e verifica strutturata delle API REST.

2 ITERAZIONE 1

In questa prima iterazione di sviluppo viene realizzata la base del sistema: la struttura architetturale (componenti, interfacce, modello dati e deployment) e le API REST che coprono il flusso operativo principale composizione e conferma degli ordini, gestione del menù, delle scorte e delle stampanti. Le versioni “intelligenti” dello smistamento delle stampe (UC5a) e della gestione predittiva delle scorte (UC5b) vengono affrontate nelle iterazioni successive.

2.1 Casi d’uso implementati

In questa iterazione vengono implementati i casi d’uso che costituiscono il nucleo operativo del sistema, descritti nelle *Fully Dressed Descriptions* dell’Iterazione 0.

Tabella 3: Casi d’uso sviluppati nell’Iterazione 1

ID	Titolo
UC1	Consultare Allergeni
UC2	Aggiungere Pietanze all’Ordine
UC3	Modificare Pietanza Ordinata
UC3b	Aggiungere Note
UC4	Gestione Ordine
UC5	Confermare Ordine
UC6	Configurare Menù
UC6a	Impostare Scorte
UC6b	Compilare Allergeni

In questa fase lo smistamento delle stampe e l’aggiornamento delle scorte alla conferma dell’ordine (UC5a e UC5b) sono realizzati nella loro forma base; l’estensione con i moduli algoritmici dedicati è oggetto delle Iterazioni 2 e 3.

2.2 Diagrammi UML

2.2.1 UML Component Diagram

Il Component Diagram rappresenta i principali componenti del sistema e le interfacce attraverso cui questi interagiscono. Il sistema è suddiviso in quattro sottosistemi: *Database*, *Application Logic*, *Client* e *Stampanti*.

Il sottosistema *Application Logic* espone quattro componenti distinti, ciascuno responsabile di una funzionalità specifica: *GestioneOrdini*, *GestioneMenu*, *GestioneScorte* e *GestioneStampe*. Ogni componente accede al database tramite la propria interfaccia dedicata (*OrdiniInterface*, *MenuInterface*, *ScorteInterface*, *StampeInterface*) ed espone a sua volta un'interfaccia verso i livelli superiori.

Il sottosistema *Client* è composto da due componenti: *CassaFrontEnd*, che utilizza *OrdiniMgmt* e *MenuLettura*, e *AdminFrontEnd*, che utilizza *MenuMgmt* e *ScorteMgmt*. L'interfaccia *MenuLettura* è separata da *MenuMgmt* in quanto la cassa necessita esclusivamente di accesso in lettura al menù, mentre l'amministratore ha la necessità di modificarne i contenuti.

Il componente *GestioneStampe* comunica con il sottosistema *Stampanti* tramite l'interfaccia *StampeMgmt*, garantendo il corretto smistamento degli ordini ai reparti di cucina, bar e griglia.

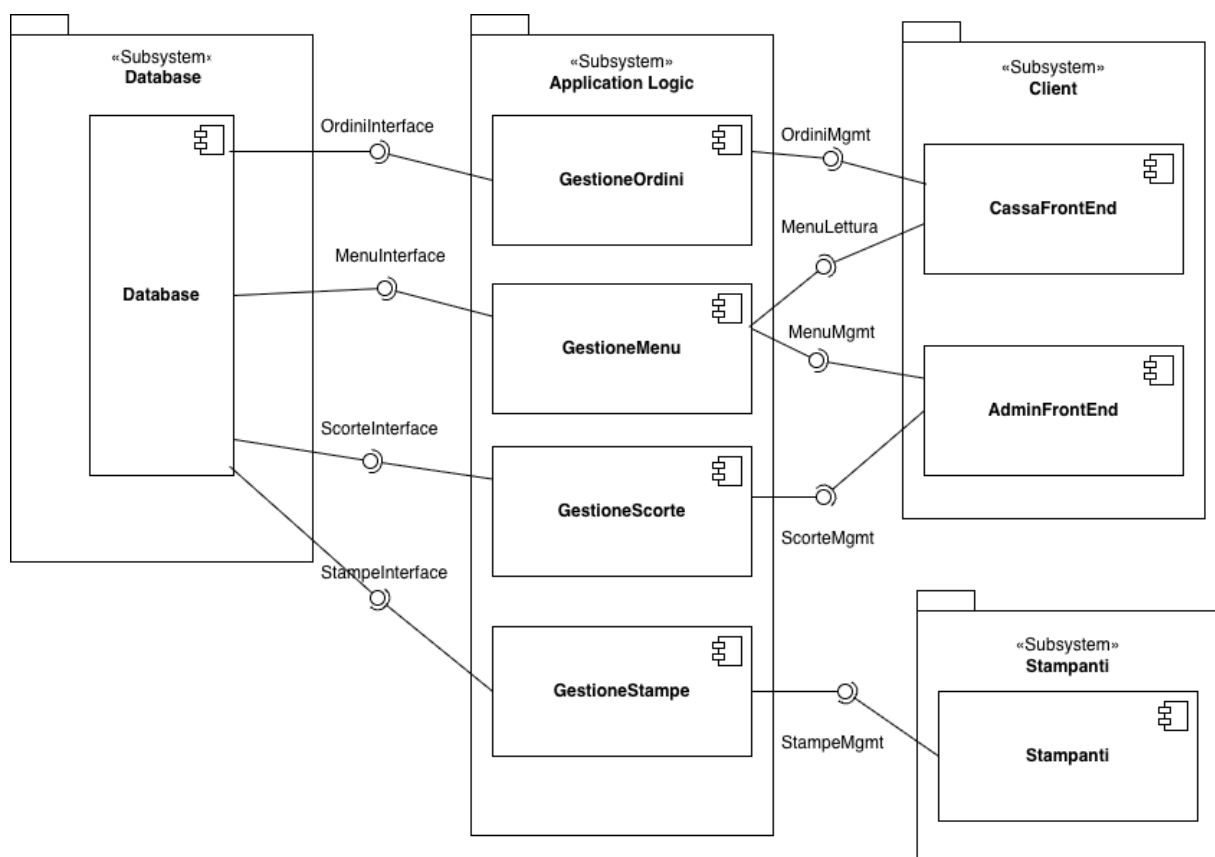


Figura 4: UML Component Diagram

2.2.2 UML Class Diagram Interface

Il Class Diagram per interfacce dettaglia le operazioni esposte da ciascuna interfaccia individuata nel Component Diagram. Ogni interfaccia rappresenta un contratto che i componenti del sistema devono rispettare, indipendentemente dalla loro implementazione concreta.

OrdiniMgmt espone le operazioni necessarie alla gestione del ciclo di vita di un ordine: creazione, modifica, eliminazione e visualizzazione. **MenuLettura** espone esclusivamente operazioni di lettura del menù, in linea con il principio del minimo privilegio applicato alla postazione cassa. **MenuMgmt** estende le funzionalità di lettura con operazioni di inserimento, modifica ed eliminazione delle voci, riservate al pannello di amministrazione. **ScorteMgmt** espone le operazioni per la consultazione e l'aggiornamento delle scorte disponibili. Infine, **StampeMgmt** espone le operazioni per l'invio degli ordini alle stampanti di reparto e per la verifica del loro stato.

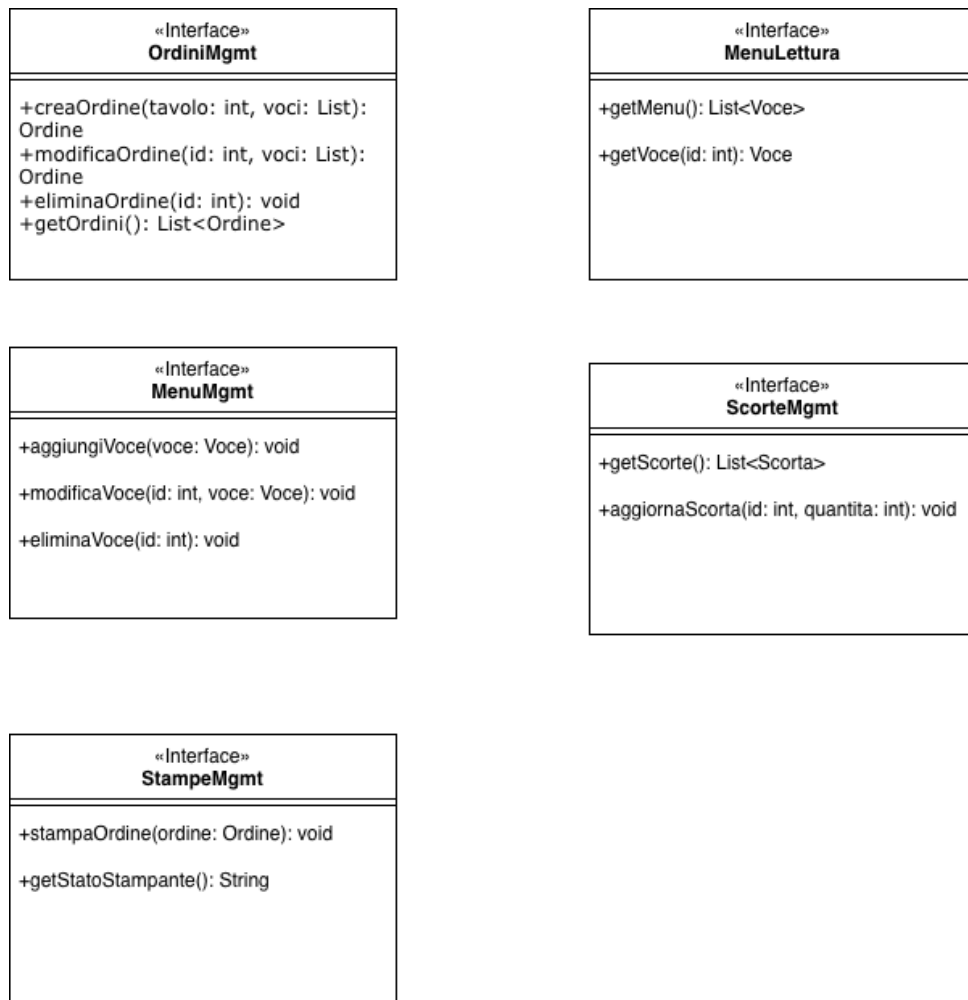


Figura 5: UML Class Diagram per interfacce

2.2.3 UML Class Diagram per tipo di dato

Il Class Diagram per tipo di dato rappresenta le entità del dominio e i relativi attributi su cui si basa la struttura del database.

Le sei classi individuate sono: **Utente**, che rappresenta l'operatore del sistema con il proprio ruolo; **Ordine**, che modella un ordine associato a un tavolo con il proprio stato e timestamp; **Voce**, che rappresenta una singola voce del menù con nome, prezzo e categoria; **Scorta**, che tiene traccia della disponibilità di un prodotto e della sua soglia minima; **Stampante**, che identifica una stampante di reparto con il proprio stato; e **Allergene**, che descrive un allergene associabile a una voce del menù.

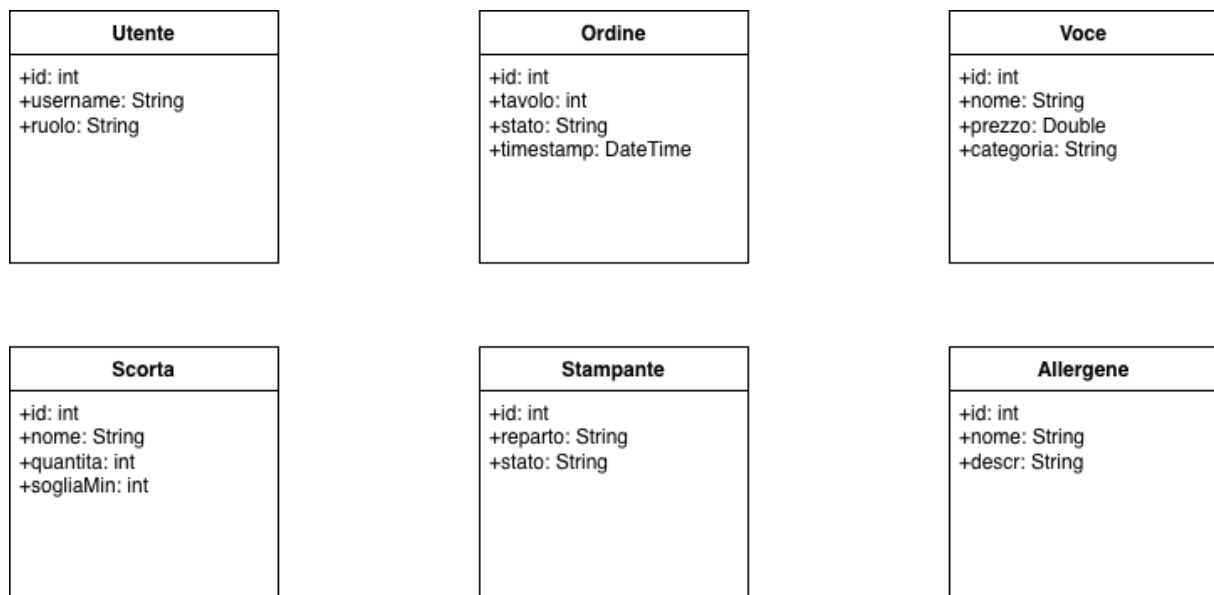


Figura 6: UML Class Diagram per tipo di dato

2.2.4 UML Deployment Diagram

Il Deployment Diagram mostra la rappresentazione hardware e software del sistema, evidenziando su quali device vengono eseguiti i componenti e attraverso quali interfacce questi comunicano tra loro.

Il nodo *MariaDB* ospita il database relazionale ed espone quattro interfacce verso il Web Server: *OrdiniInterface*, *MenuInterface*, *ScorteInterface* e *StampeInterface*. Il nodo *Web Server* contiene i quattro componenti applicativi *GestioneOrdini*, *GestioneMenu*, *GestioneScorte* e *GestioneStampe*, ciascuno marcato come «Control». Il nodo *Client* ospita i due componenti «Boundary» *CassaFrontEnd* e *AdminFrontEnd*, raggiungibili tramite le interfacce *OrdiniMgmt*, *MenuLettura*, *MenuMgmt* e *ScorteMgmt*. Infine, il nodo *Stampanti* contiene il componente «Boundary» *Stampanti*, collegato al server tramite l'interfaccia *StampeMgmt*.

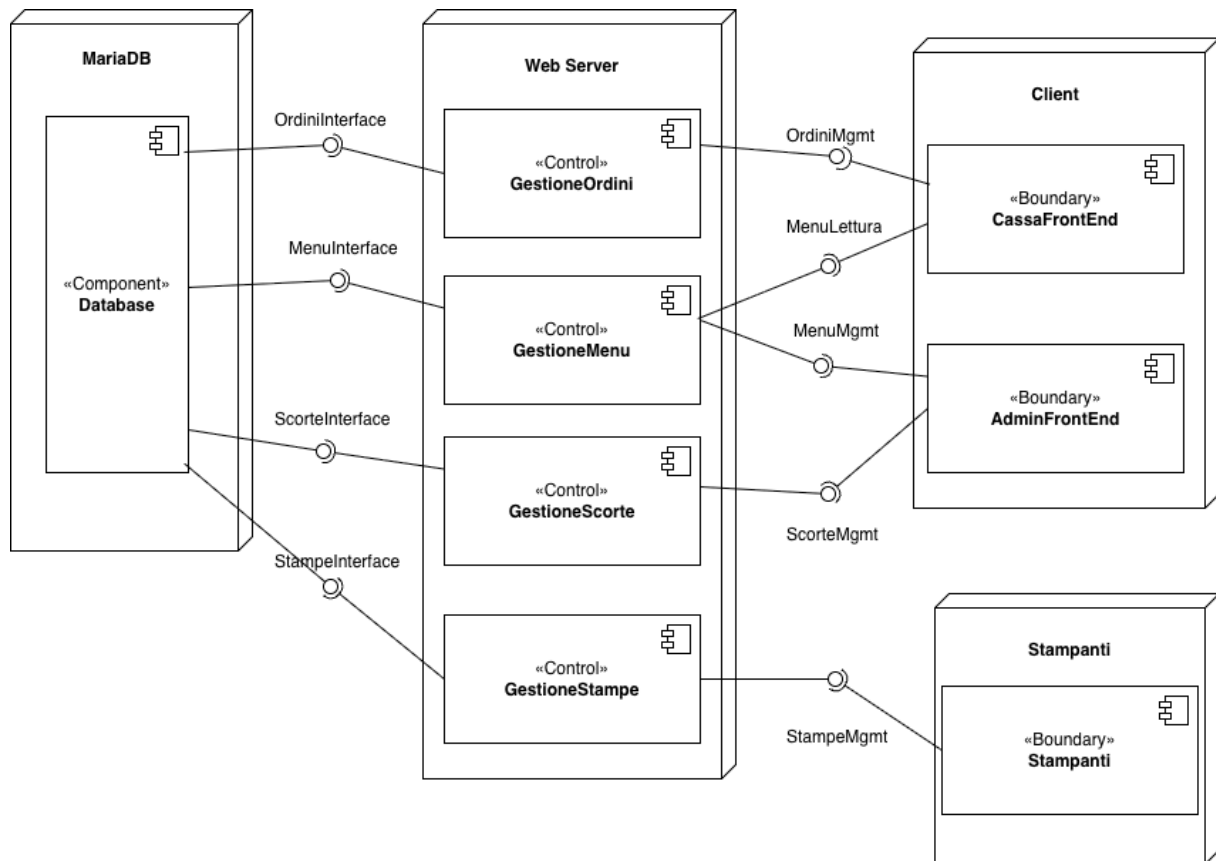


Figura 7: UML Deployment Diagram

2.3 Test

2.3.1 Analisi statica del codice

A conclusione dell'iterazione, il backend di base è stato sottoposto ad analisi statica con la regola `complexity` di **ESLint** (complessità ciclomatica) e con **madge** (accoppiamento tra moduli e dipendenze circolari). Questa fotografia stabilisce la *baseline* del sistema prima dell'introduzione dei moduli algoritmici rispetto a cui verranno misurati gli incrementi delle Iterazioni 2 e 3.

Tabella 4: Metriche statiche del backend di base (Iterazione 1)

Metrica (backend, <code>src/</code>)	Valore
Complessità ciclomatica massima	51 (<code>parsaFlusso</code>)
Funzioni con complessità ciclomatica > 15	4
Funzioni totali analizzate	67
Moduli accoppiati a <code>db.js</code> (C_a)	7
Dipendenze circolari	0
Righe di codice (SLOC, escluse vuote e commenti)	1 506

Il sistema di base è strutturalmente sano: nessuna dipendenza circolare e un accoppiamento al database contenuto. L'unico punto realmente critico è il parser del flusso ESC/POS `parsaFlusso` (complessità 51), seguito dal bootstrap del server; tutte le altre funzioni restano ampiamente sotto la soglia di 15, come mostra la Figura 8.

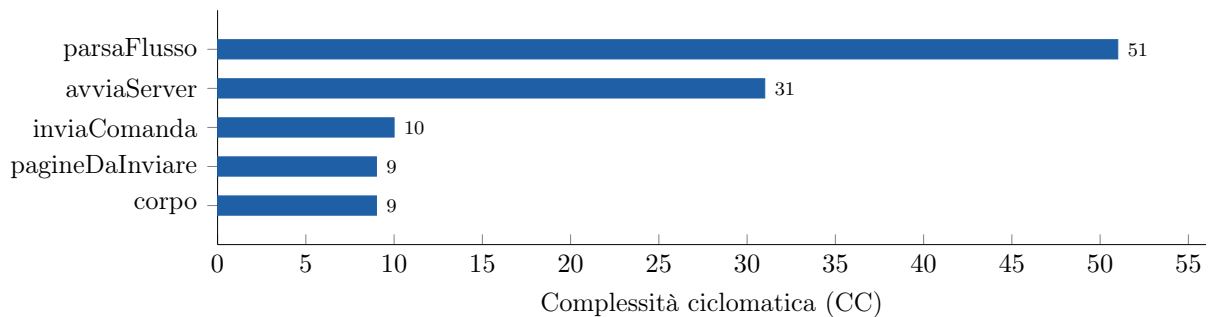


Figura 8: Complessità ciclomatica delle funzioni più impegnative del backend di base. Solo `parsaFlusso` (parser ESC/POS) e il bootstrap del server (`avviaServer`) superano la soglia di 15.

L'accoppiamento tra i moduli e la loro dimensione completano il quadro: i moduli più connessi sono il bootstrap `index.js` e la catena di stampa (Figura 9); i file più estesi restano sotto le 280 righe (Figura 10).

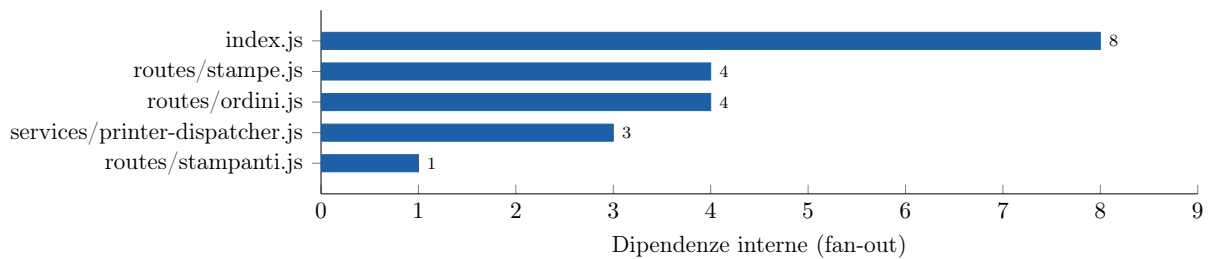


Figura 9: Accoppiamento: numero di dipendenze interne (fan-out) dei moduli più connessi. Complessivamente 7 moduli su 10 dipendono da `db.js`; nessuna dipendenza circolare.

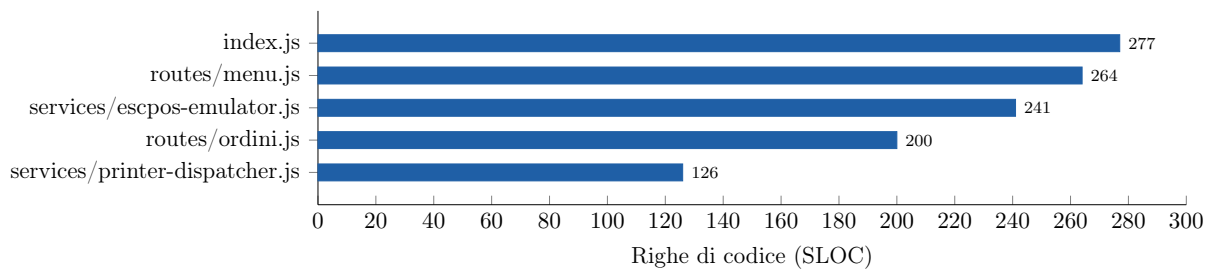


Figura 10: Dimensione dei moduli più grandi del backend di base, in righe di codice (SLOC).

2.3.2 Test automatizzati

Le funzionalità di base del backend (gestione di ordini, menù, scorte e stampanti) sono coperte da una suite di test unitari con **Vitest**, eseguibile con `npm test`. La suite conta **47 test**, tutti superati, distribuiti sui quattro moduli CRUD; l'esito reale dell'esecuzione è riportato in Figura 11.

```
C:\Users\masig\Documents\GitHub\Progetto-PAC\CODE\gestionale-feste\backend>npm test

> gestionale-feste-backend@1.0.0 test
> vitest run

 RUN v4.1.9 C:/Users/masig/Documents/GitHub/Progetto-PAC/CODE/gestionale-feste/backend

 ✓ __tests__/scorte.test.js (10 tests) 46ms
 ✓ __tests__/ordini.test.js (12 tests) 66ms
 ✓ __tests__/menu.test.js (18 tests) 85ms
 ✓ __tests__/stampanti.test.js (7 tests) 20ms

Test Files  4 passed (4)
Tests      47 passed (47)
Start at   17:42:08
Duration   3.77s (transform 193ms, setup 0ms, import 4.72s, tests 218ms, environment 2ms)
```

Figura 11: Esecuzione della suite di test del sistema base con Vitest: 47/47 test superati sui moduli Ordini, Menu, Scorte e Stampanti.

I test sono organizzati per risorsa:

1. **Ordini** (12 test). Verificano la creazione, la consultazione e la conferma dell'ordine, con calcolo del totale e decremento delle scorte.
2. **Menu** (18 test). Coprono la gestione del catalogo: voci visibili e nascoste, pietanze aggregate, settori di visualizzazione e associazione degli allergeni.
3. **Scorte** (10 test). Validano il calcolo dello stato visivo secondo le soglie configurate, l'aggiornamento (upsert) della disponibilità e il rifornimento.
4. **Stampanti** (7 test). Verificano la configurazione delle stampanti di reparto: creazione, aggiornamento e rimozione.

2.3.3 API esposte

Il backend espone quattro risorse principali: *Ordini*, *Menu*, *Scorte* e *Stampanti*. Per tutte valgono le stesse convenzioni, valide anche per gli endpoint introdotti nelle iterazioni successive: la base URL è `http://localhost:3001/api`, lo scambio dati avviene in JSON e gli errori restituiscono sempre `{"errore": "...}"` con codice HTTP 400, 404, 409 o 500. Le operazioni di scrittura (**POST**, **PUT**) sono verificate tramite Postman, di cui si riporta richiesta e risposta.

Ordini

Gestisce il ciclo di vita degli ordini: creazione, consultazione e conferma con calcolo del totale e aggiornamento delle scorte.

GET /api/ordini

Restituisce tutti gli ordini del giorno corrente con righe e note per porzione.

```
[
  {
    "id": 1,
    "stato": "confermato",
    "totale": "24.50",
    "importo_pagato": "30.00",
    "timestamp": "2026-04-16T12:30:00.000Z",
    "righe": [
      {
        "voce_id": 3, "quantita": 2, "nome": "Lasagne al forno", "prezzo": "8.50",
        "note": [ { "numero_porzione": 1, "testo": "Senza besciamella" } ]
      }
    ]
  }
]
```

GET /api/ordini/:id

Dettaglio completo di un singolo ordine. Include `settore_stampa` per ogni riga, usato per l'instradamento alla stampante corretta. Risposta 404 se non trovato.

```
{
  "id": 1, "stato": "confermato", "totale": "24.50",
  "righe": [
    {
      "voce_id": 3, "quantita": 2, "nome": "Lasagne al forno",
```

```

    "prezzo": "8.50", "settore_stampa": "cucina",
    "note": [ { "numero_porzione": 1, "testo": "Senza besciamella" } ]
  }
]
}

```

POST /api/ordini

Crea un ordine in stato `in_composizione`. Richiede almeno una riga con `voce_id` e `quantita`. Va seguito dalla conferma per finalizzarlo.

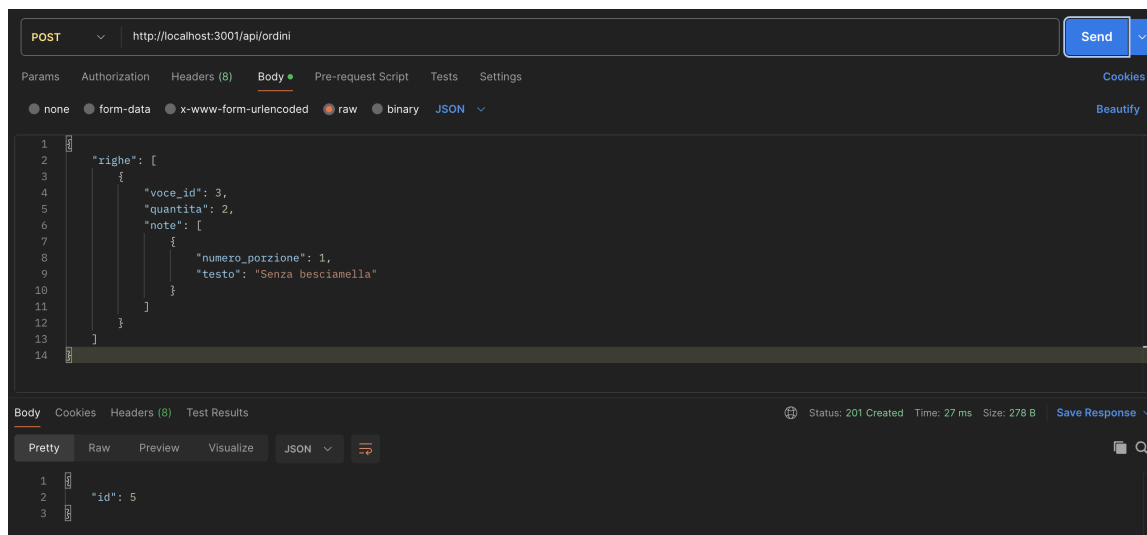


Figura 12: Postman — POST /api/ordini

POST /api/ordini/:id/conferma

Conferma l'ordine: calcola il totale, decrementa le scorte e porta lo stato a `confermato`. Emette l'evento WebSocket `scorte_aggiornate`.

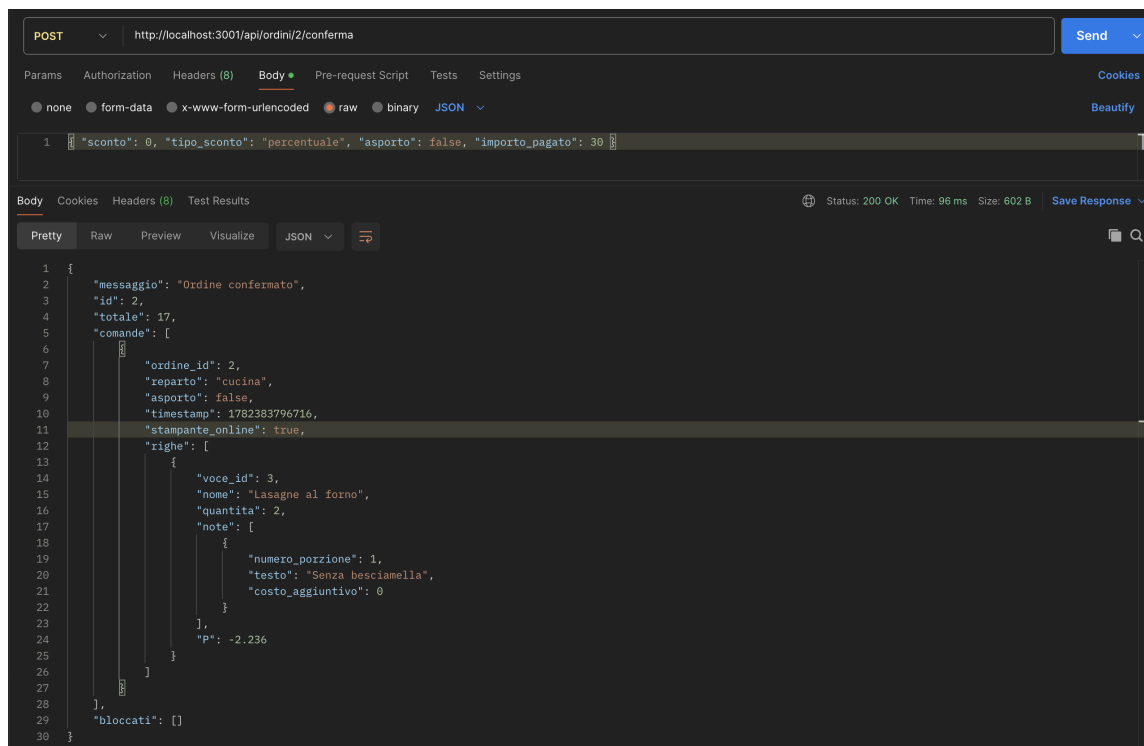


Figura 13: Postman — POST `/api/ordini/:id/conferma`

Menu

Gestisce il catalogo delle voci, incluse pietanze aggregate (composte da più componenti), settori di visualizzazione e allergeni.

GET `/api/menu`

Voci visibili (`visibile = 1`) con dati di scorta. Usato dalla cassa per popolare la griglia prodotti.

```
[
  {
    "id": 1, "codice": "P001", "nome": "Risotto ai funghi", "prezzo": "9.00",
    "categoria": "Primo", "settore_visualizzazione": "Primi",
    "settore_stampa": "cucina", "colore_tasto": "#E8A838",
    "visibile": 1, "asportabile": 1,
    "scorta_quantita": 45, "soglia_giallo": 10, "soglia_rosso": 3, "scorta_attiva": 1
  }
]
```

GET `/api/menu/tutte`

Come `/api/menu` ma include le voci nascoste e il campo `num_componenti` per identificare le pietanze aggregate. Usato dal pannello Admin.

POST `/api/menu`

Crea una nuova voce. Il codice è auto-generato (`<iniziale-categoria><progressivo>`, es. P007). Campo obbligatorio: `nome`.

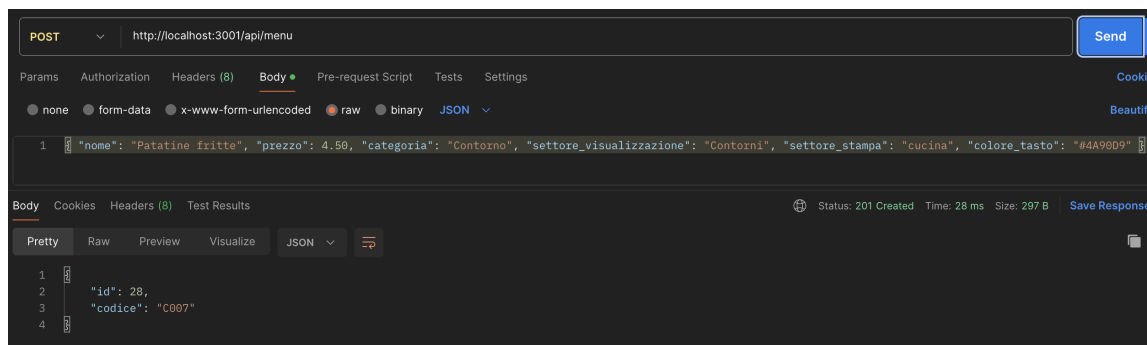


Figura 14: Postman — POST /api/menu

PUT /api/menu/:id

Aggiornamento parziale: si inviano solo i campi da modificare. Modificabili: nome, prezzo, categoria, settore_visualizzazione, settore_stampa, colore_tasto, ordine_schermo, visibile, asportabile, modalita_stampa.

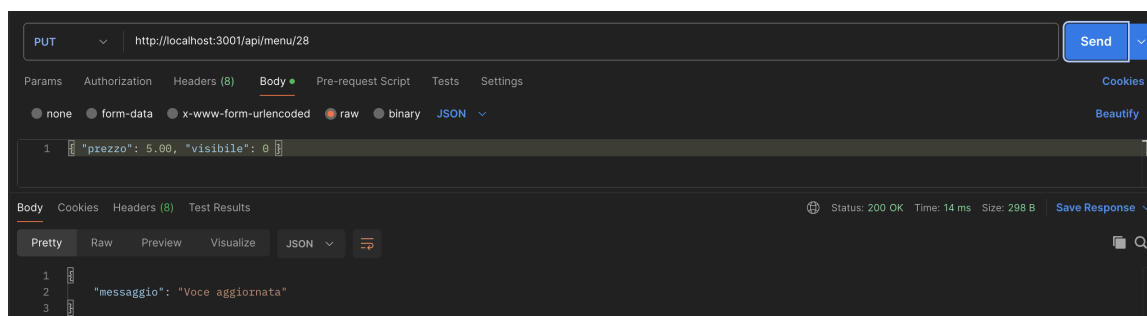


Figura 15: Postman — PUT /api/menu/:id

DELETE /api/menu/:id

Elimina una singola voce. Operazione irreversibile.

Scorte

Gestisce la disponibilità in tempo reale, con stato visivo calcolato automaticamente secondo le soglie configurate per voce.

Condizione	stato_visivo
quantita = 0	esaurito
quantita <= soglia_rosso (default 3)	critico
quantita <= soglia_giallo (default 10)	attenzione
altrimenti	disponibile

GET /api/scorte

Tutte le scorte attive con stato visivo calcolato.

```
[
  {
```

```

    "voce_id": 1, "nome": "Risotto ai funghi", "codice": "P001",
    "quantita": 8, "soglia_giallo": 10, "soglia_rosso": 3,
    "attiva": 1, "stato_visivo": "attenzione"
  }
]

```

GET /api/scorte/:voce_id

Scorta di una singola voce. Ritorna null se non configurata.

PUT /api/scorte/:voce_id

Imposta o sovrascrive (upsert) la scorta. Richiede **quantita**. Emette **scorte_aggiornate** via WebSocket.

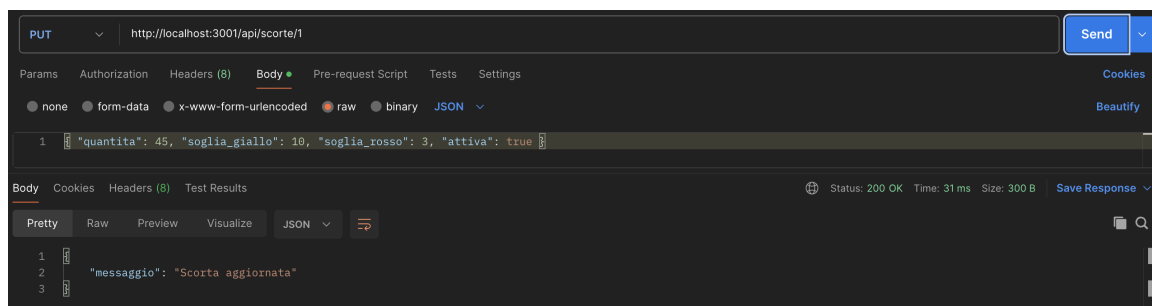


Figura 16: Postman — PUT /api/scorte/:voce_id

POST /api/scorte/:voce_id/rifornimento

Aggiunge unità alla scorta e registra l'operazione nello storico. La **quantita** deve essere maggiore di zero. Emette **scorte_aggiornate** via WebSocket.

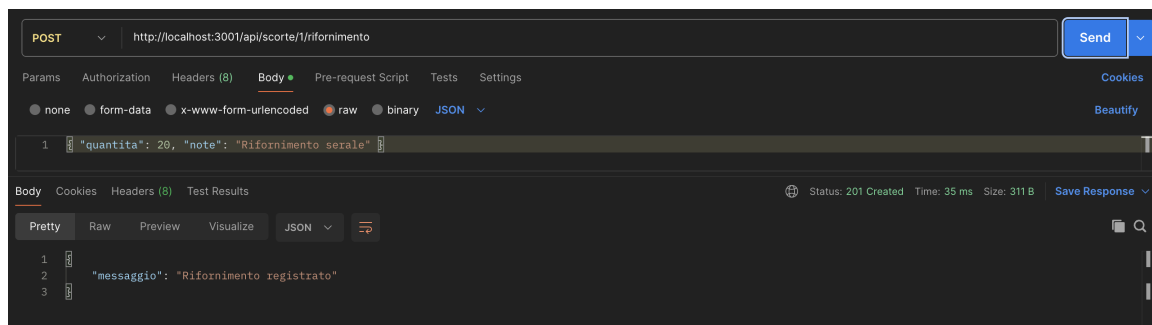


Figura 17: Postman — POST /api/scorte/:voce_id/rifornimento

Stampanti

Gestisce le stampanti ESC/POS in rete locale, raggruppate per reparto (cucina, bar, griglia).

GET /api/stampanti

Lista di tutte le stampanti configurate.

```

[
  { "id": 1, "reparto": "cucina", "indirizzo_ip": "192.168.1.101", "porta": 9100, "
    stato": "sconosciuto" }
]

```

]

POST /api/stampanti

Aggiunge una nuova stampante. Obbligatori: reparto e indirizzo_ip. Porta default: 9100.

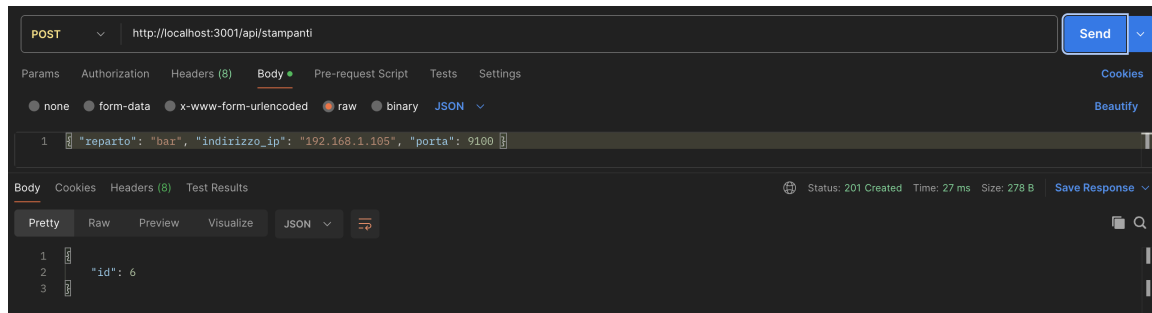


Figura 18: Postman — POST /api/stampanti

PUT /api/stampanti/:id

Aggiorna i dati di una stampante. Body identico al POST.

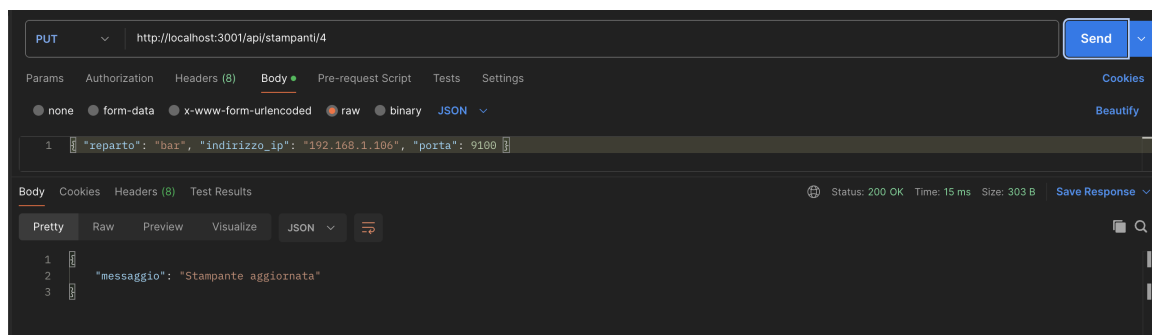


Figura 19: Postman — PUT /api/stampanti/:id

DELETE /api/stampanti/:id

Rimuove una stampante dalla configurazione.

3 ITERAZIONE 2

In questa iterazione il sistema viene esteso con il primo dei due moduli algoritmici del progetto: lo **Smistatore Intelligente Multi-Reparto**, che introduce un instradamento adattivo delle comande verso i reparti di preparazione. L'iterazione comprende l'evoluzione dei diagrammi UML (con il nuovo sottosistema di smistamento e i nuovi componenti di gestione), la descrizione e l'analisi dell'algoritmo, la suite di test dedicata, le API esposte e una prima rilevazione delle metriche statiche del codice.

3.1 Casi d'uso implementati

Tabella 5: Casi d'uso sviluppati nell'Iterazione 2

ID	Titolo	Relazione
UC7	Smistatore Intelligente Multi-Reparto	realizza UC5a in forma avanzata

Il caso d'uso UC5a (*Genera Stampe e Smistare*), descritto nell'Iterazione 0 e implementato nella sua forma base, viene qui realizzato in forma avanzata dal modulo algoritmico UC7. Per non sovraccaricare lo stesso identificatore con due significati, il modulo riceve un ID dedicato.

3.2 Diagrammi UML

I diagrammi seguenti aggiornano quelli dell'Iterazione 1 per riflettere l'introduzione del sottosistema di smistamento e dei nuovi componenti di gestione. Per ciascun diagramma vengono evidenziate le differenze rispetto alla versione precedente.

3.2.1 UML Component Diagram

Rispetto all'Iterazione 1, l'architettura è stata ampliata e si articola ora in cinque sottosistemi principali: *Database*, *Application Logic*, *Client*, *Stampanti* e il nuovo sottosistema *Smistamento*.

Il sottosistema *Application Logic* è organizzato in sei componenti. Accanto a *GestioneOrdini*, *GestioneMenu*, *GestioneScorte* e *GestioneStampe* sono formalizzate due nuove viste logiche: *GestioneAllergeni* e *GestioneConfigurazione*. La separazione è concettuale, finalizzata a distinguere la sola lettura sugli allergeni dall'amministrazione del menù; a livello implementativo entrambe confluiscono nel modulo `routes/menu.js`, condividendo lo stesso accesso al database. Il diagramma rappresenta pertanto tali componenti come viste sulla medesima sorgente dati, senza introdurre interfacce di persistenza fisicamente distinte.

Nel sottosistema *Client* le interfacce sono state aggiornate per riflettere questa separazione di responsabilità. L'interfaccia di scrittura *MenuMgmt* viene dismessa e sostituita dalla nuova interfaccia *ConfigurazioneMgmt*, riservata al componente *AdminFrontEnd*, che assorbe la logica amministrativa relativa a inserimento, modifica ed eliminazione delle voci, impostazione delle scorte iniziali, configurazione delle stampanti e associazione degli

allergeni. La *CassaFrontEnd* mantiene l'interfaccia di sola lettura *MenuLettura* per la visualizzazione della griglia prodotti e ottiene accesso a *ScorteMgmt*, prima non disponibile. L'interfaccia *AllergeniMgmt*, esposta dalla vista logica *GestioneAllergeni*, è condivisa da entrambi i client in modalità di sola lettura.

Infine, il flusso di stampa ha subito una revisione architetturale. *GestioneStampe* non comunica più direttamente con il sottosistema *Stampanti*: è stato introdotto il sottosistema *Smistamento*, contenente il componente *SistemaSmistamento*, che funge da dispatcher instradando le richieste verso i componenti *Stampante* tramite l'interfaccia *StampeMgmt*. La comunicazione tra *GestioneStampe* e *SistemaSmistamento* avviene attraverso l'interfaccia *SmistamentoMgmt*. Questa soluzione rende l'instradamento verso i reparti più scalabile e disaccoppiato rispetto all'approccio precedente.

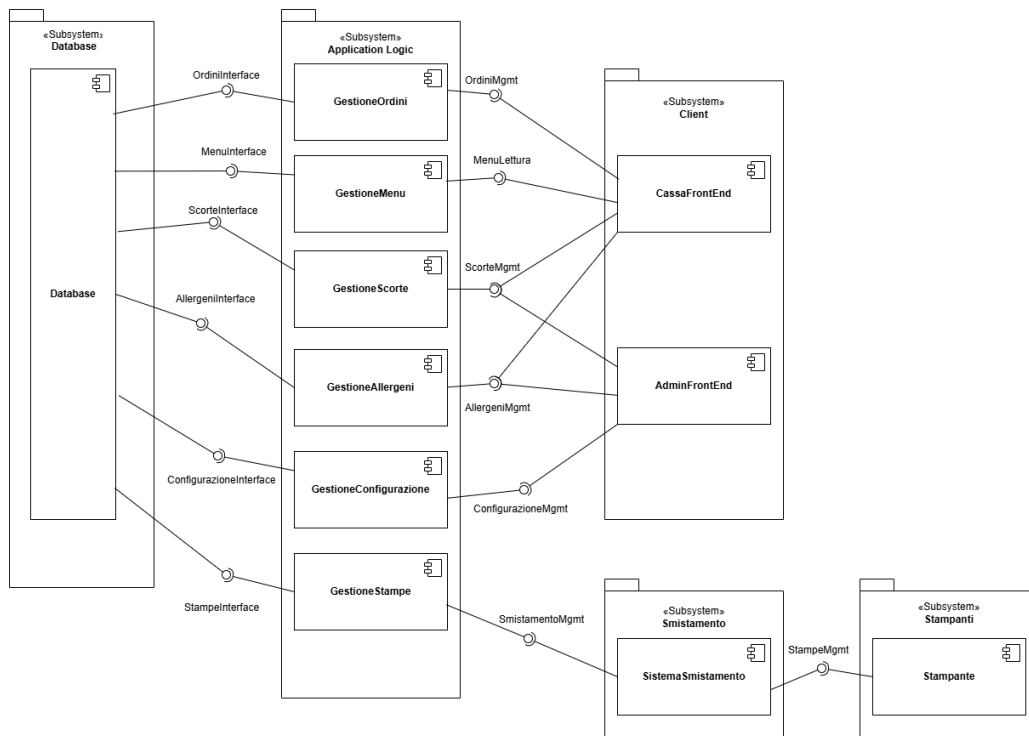


Figura 20: Diagramma dei Componenti UML Iterazione 2

3.2.2 UML Class Diagram Interface

OrdiniMgmt espone le operazioni per la gestione operativa e il ciclo di vita di una comanda: inserimento, rimozione e modifica quantitativa delle pietanze, inserimento di note libere per la cucina, gestione delle specifiche di asporto, applicazione di sconti, calcolo del resto e procedure di conferma o annullamento dell'ordine.

MenuLettura funge da modulo di sola consultazione ad uso della cassa, esponendo metodi per l'ottenimento del menù e il filtraggio delle voci per categoria o per settore. L'intera logica amministrativa e di scrittura è invece delegata alla nuova interfaccia *ConfigurazioneMgmt*, riservata al backoffice, che espone inserimento, modifica ed eliminazione fisica delle pietanze, impostazione delle scorte iniziali, configurazione e recupero delle stampanti di reparto e associazione degli allergeni ai singoli piatti.

La nuova interfaccia **AllergeniMgmt** rende fruibili le informazioni sulle intolleranze alimentari, con metodi per consultare e stampare il report degli allergeni sia per una specifica pietanza sia a livello globale per l'intero menù, su entrambi i front-end.

L'interfaccia **ScorteMgmt** è stata potenziata per garantire la consistenza dell'inventario durante il servizio: oltre a verificare la disponibilità dei piatti, espone i metodi per decrementare le scorte residue e per gestire i contatori aggregati associati ai piatti a disponibilità limitata.

Infine, l'invio alle stampanti è stato disaccoppiato tramite l'interfaccia **SmistamentoMgmt**, legata al rispettivo sottosistema: essa si occupa dell'elaborazione ad alto livello dei flussi di stampa (generazione e instradamento dei biglietti verso i reparti corretti, produzione della copia cliente, notifica dei piatti pronti per l'asporto). A valle, l'interfaccia **StampeMgmt** gestisce la comunicazione a basso livello con i dispositivi fisici, esponendo i metodi per l'invio del file di stampa e per il controllo dello stato hardware delle stampanti.

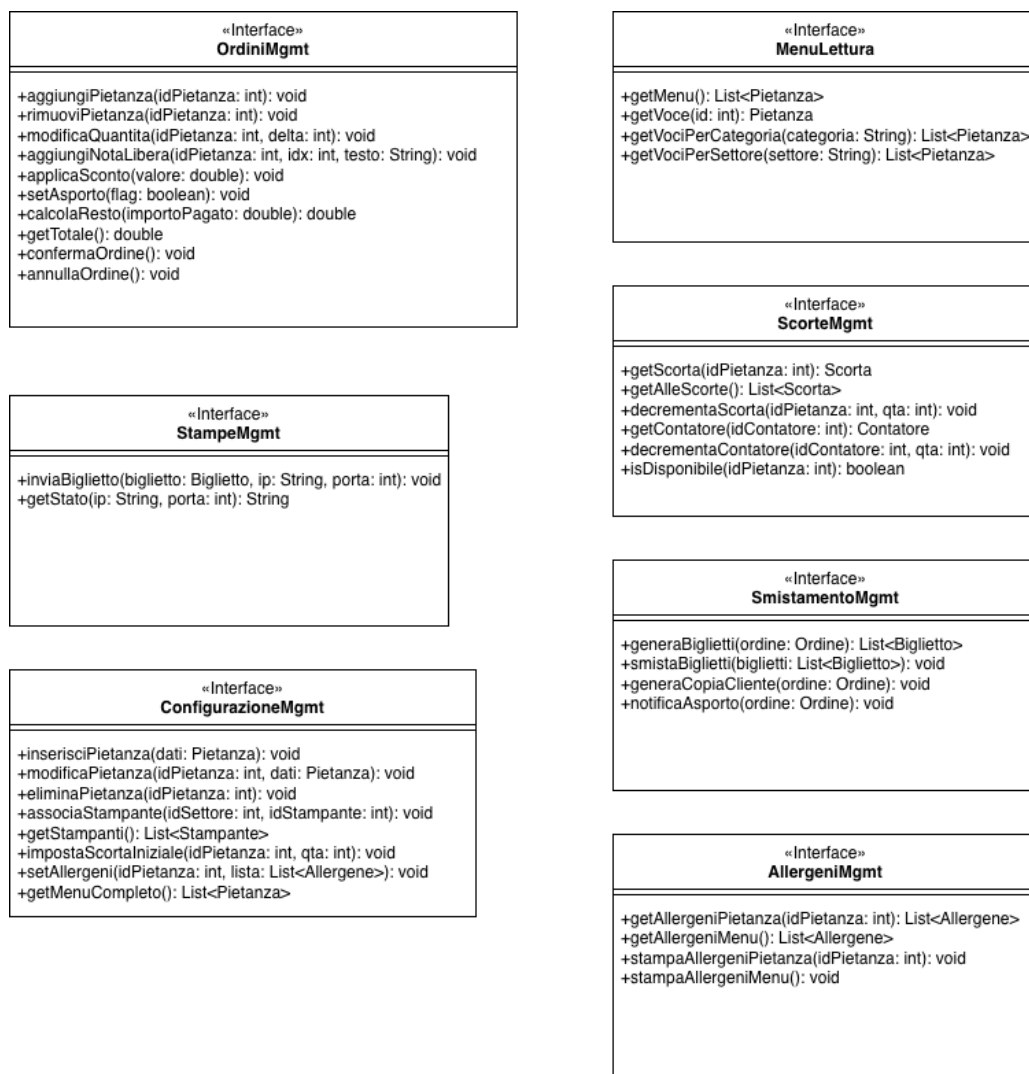


Figura 21: Diagramma UML delle Classi per interfacce Iterazione 2

3.2.3 UML Class Diagram per tipo di dato

Il Class Diagram per tipo di dato rappresenta le entità del dominio e i relativi attributi, in corrispondenza diretta con lo schema relazionale definito in `database/init.sql`: nomi e attributi coincidono con quelli effettivi delle tabelle, così che diagramma e database descrivano la stessa struttura. Le entità sono raggruppate per ambiti logici.

Menù. L'entità *Voce* costituisce il cuore del catalogo, con `codice`, `nome`, `prezzo`, `categoria`, `settore_visualizzazione`, `settore_stampa`, `colore_tasto`, `visibile`, `asportabile`, `modalita_stampa`, `copia_scontrino_cliente` e i parametri usati dai moduli algoritmici: `tempo_preparazione`, `tempo_riapprovvigionamento` e `priorita_voce`. Le pietanze aggregate (menù composti) sono modellate dalla relazione *VoceComposizione* (`voce_aggregata_id`, `voce_componente_id`).

Disponibilità. La classe *Scorta* (`voce_id`, `quantita`, `soglia_giallo`, `soglia_rosso`, `attiva`) mantiene la disponibilità per singola voce, con le soglie di allerta come propri attributi; *ScortaStorico* (`voce_id`, `quantita`, `data_rifornimento`, `note`) registra i rifornimenti. Gli ingredienti condivisi sono gestiti da *ContatoreAggregato* (`nome`, `quantita`) e dalla relazione *VoceContatore* (`voce_id`, `contatore_id`).

Comande. L'entità *Ordine* (`tavolo`, `stato`, `asporto`, `sconto`, `tipo_sconto`, `totale`, `importo_pagato`, `timestamp`, `utente_id`) traccia stato e dati fiscali; *OrdineRiga* (`ordine_id`, `voce_id`, `quantita`) mappa le singole quantità; *OrdineRigaNota* (`riga_id`, `numero_porzione`, `testo`, `costo_aggiuntivo`) gestisce le note per porzione e *NotaPreimpostata* (`voce_id`, `testo`, `costo_aggiuntivo`) le varianti rapide.

Allergeni. L'entità *Allergene* (`nome`, `descr`) è associata alle voci tramite la relazione *VoceAllergene* (`voce_id`, `allergene_id`).

Stampa. L'entità *Stampante* (`reparto`, `nome`, `indirizzo_ip`, `porta`, `modello`, `primaria`, `attiva`, `stato`) descrive i dispositivi di reparto, mentre *StampaEseguita* (`ordine_id`, `stampante_id`, `reparto`, `payload`, `esito`, `durata_ms`, `timestamp`) costituisce l'audit log delle stampe inviate.

Configurazione. L'entità *Configurazione* (`chiave`, `valore`) conserva i parametri runtime usati dai moduli algoritmici (es. soglie e finestre temporali).

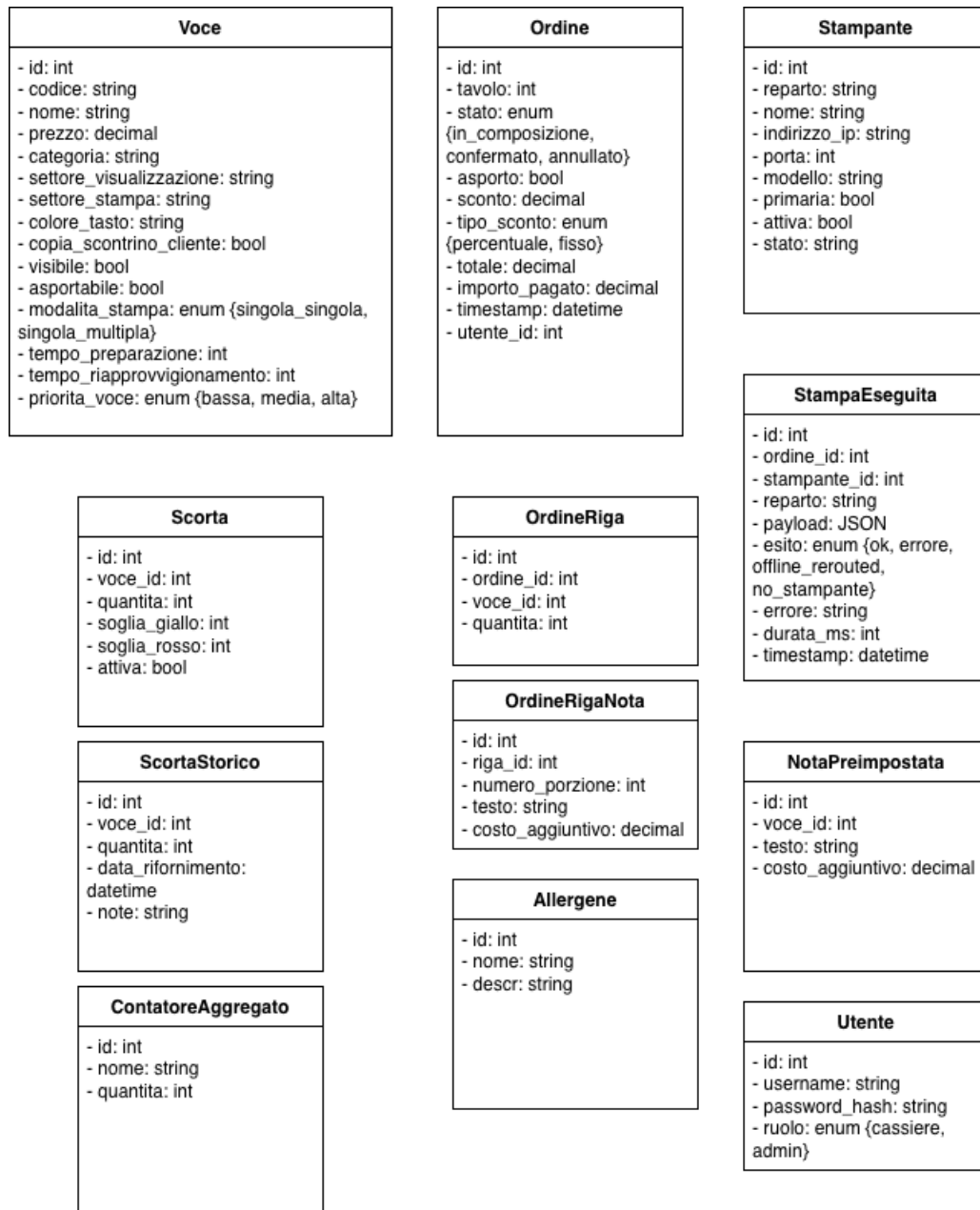


Figura 22: Diagramma UML di Classe per tipo di dato Iterazione 2

3.2.4 UML Deployment Diagram

Il nodo *FesteDB* ospita il database relazionale e funge da sorgente dati centrale. Il nodo centrale *Web Server* ospita il core applicativo, organizzato nei moduli di gestione (*GestioneOrdini*, *GestioneMenu*, *GestioneScorte*, *GestioneAllergeni*, *GestioneConfigurazione* e *GestioneStampe*); allergeni e configurazione condividono il modulo *routes/menu.js* e quindi lo stesso accesso al database. All'interno del nodo si nota una dipendenza logica tra *GestioneMenu* e *GestioneScorte*.

Il nodo *Client* ospita le applicazioni di frontend, *WebAppCassa* e *WebAppBackOffice*. La comunicazione tra Client e Web Server è mediata dalle interfacce aggiornate: la cassa accede a *OrdiniMgmt*, *MenuLettura*, *ScorteMgmt* e *AllergeniMgmt*; il backoffice si affida anch'esso ad *AllergeniMgmt* e utilizza in modo esclusivo *ConfigurazioneMgmt* per l'amministrazione centralizzata.

Infine, l'infrastruttura di stampa si divide su due nodi logici distinti. Il Web Server comunica tramite l'interfaccia *SmistamentoMgmt* con il nuovo nodo *Smistamento*, che ospita il componente *SistemaSmistamento* delegato alla logica di routing. Questo nodo comunica a sua volta con i dispositivi fisici allocati nel nodo *Stampante* tramite l'interfaccia a basso livello *StampeMgmt*.

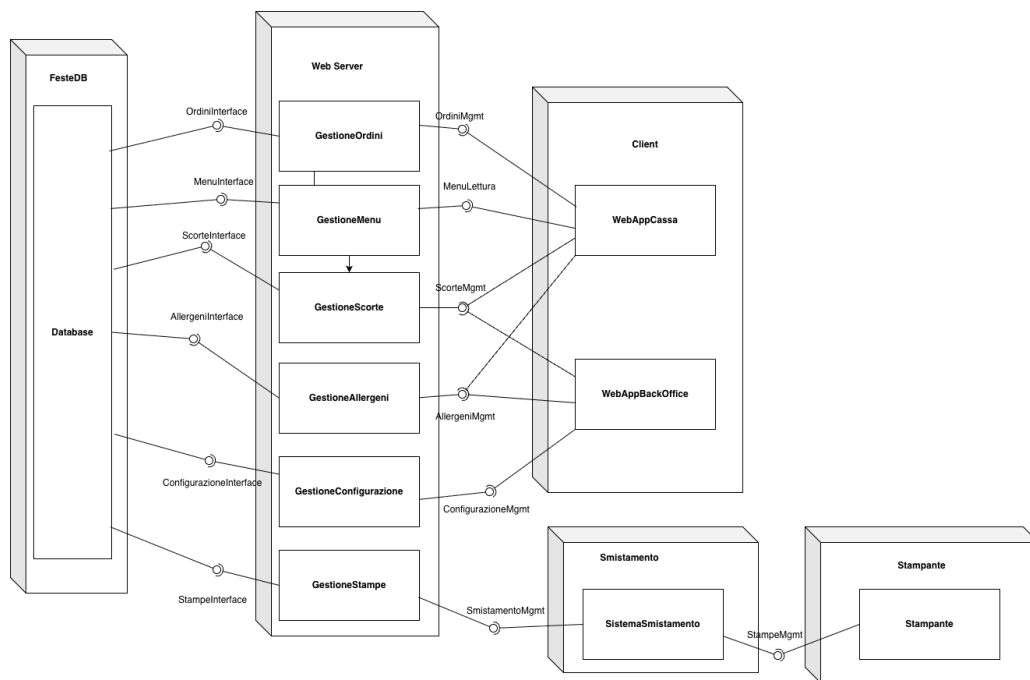


Figura 23: Diagramma UML di Deployment Iterazione 2

3.3 Algoritmo: Smistatore Intelligente Multi-Reparto

Introduzione

L'algoritmo **Smistatore Intelligente Multi-Reparto** bilancia il carico operativo tra i reparti di un sistema di gestione ordini per la ristorazione.

Nei sistemi POS tradizionali, ogni voce d'ordine viene instradata verso un reparto fisso (`settore_stampa`), indipendentemente dal carico corrente. Questo genera due problemi:

- **Saturazione asimmetrica:** tutti gli ordini di un reparto confluiscono su di esso anche quando è sovraccarico.
- **Mancanza di adattività:** il sistema ignora il tempo di preparazione residuo, la quantità richiesta, lo stato delle stampanti e la disponibilità delle scorte.

L'algoritmo introduce un modello di scoring dinamico che valuta, per ogni task, il reparto ottimale compatibile in base al carico reale istantaneo (Figura 26).

Attori coinvolti

- **Sistema POS:** emette gli ordini con i relativi parametri (timestamp, priorità, quantità).
- **Reparti operativi:** cucina, bar, griglia destinatari delle comande.
- **Stampanti / Dashboard:** destinatari fisici; il sistema gestisce fallback se offline.
- **Cassa:** riceve notifiche di blocco per scorta insufficiente.

Trigger e postcondizioni

Trigger: Ricezione di un ordine con una o più righe (pietanze) e relativi metadati.

Postcondizione: Per ogni riga valida: assegnazione al reparto con carico minore compatibile, generazione della comanda separata per reparto, invio alla stampante o dashboard corretta.

Parametri di Input

Ogni riga d'ordine viene convertita in un *task di preparazione*. La Tabella 6 riporta i parametri considerati per routing e ranking.

Tabella 6: Parametri di input dell'algoritmo

Parametro	Descrizione
settore_stampa	Reparto primario configurato nel menu; punto di partenza per i reparti compatibili.
numero_task_in_coda	Task in attesa per ogni reparto. Componente principale del carico.
tempo_preparazione	Tempo medio stimato di preparazione (secondi).
quantità	Porzioni richieste. Incide sul carico e sul raggruppamento batch.
timestamp	Marca temporale di arrivo; usata per urgenza e ordinamento.
stato_stampante	Online/offline. Attiva il fallback se offline.
disponibilità_scorte	Flag o quantità residua. Se insufficiente, il task viene bloccato.
priorità_ordine	Categoria: tavolo, asporto, urgenza. Peso nel ranking P .

Passi dell'Algoritmo

L'algoritmo si articola in undici passi sequenziali, con diramazioni per la gestione dei casi limite. L'obiettivo è minimizzare il carico massimo tra i reparti e massimizzare la reattività.

Passo 1 Ricezione e lettura righe ordine All'arrivo di un ordine, il sistema estrae da ogni riga il `settore_stampa`, il `timestamp`, la `priorità` e la `quantità`. Ogni riga diventa un task di preparazione indipendente.

Passo 2 Conversione in task di preparazione Ogni riga viene normalizzata in un oggetto task con struttura uniforme, indipendentemente dal tipo di pietanza o reparto.

Passo 3 Verifica disponibilità scorte Prima del routing, il sistema verifica che la scorta sia sufficiente per la quantità richiesta. Questo *guard* evita di occupare capacità di reparto inutilmente. Se la scorta è insufficiente, il task viene bloccato con notifica alla cassa; le restanti righe proseguono normalmente.

Passo 4 Identificazione reparti compatibili Per ogni task con scorta sufficiente, il sistema individua i reparti in grado di gestire la pietanza: il reparto primario (`settore_stampa`) più eventuali alternative configurate (es. una pietanza frita gestibile sia da cucina che da griglia).

Passo 5 Calcolo del carico per ogni reparto Per ciascun reparto compatibile viene calcolato un punteggio di carico:

$$\text{carico_reparto} = n_{\text{task}} \times t_{\text{prep_residuo}} \times \alpha_{\text{stress}} \times \beta_{\text{stampante}} \quad (1)$$

dove n_{task} è il numero di task in coda, $t_{\text{prep_residuo}}$ è la somma del tempo residuo di tutti i task (secondi), $\alpha_{\text{stress}} \in [1.0, 1.5]$ è il moltiplicatore per reparto in stress e $\beta_{\text{stampante}} \in [1.0, 2.0]$ è il moltiplicatore per stampante offline. Nell'implementazione, $\alpha_{\text{stress}} = 1.3$ si attiva oltre la soglia di 5 task in coda e $\beta_{\text{stampante}} = 2.0$ a stampante offline.

Passo 6 Selezione del reparto con carico minimo Se il task è compatibile con più reparti, il sistema sceglie quello con `carico_reparto` più basso (selezione greedy). In caso di parità, prevale il reparto con il timestamp dell'ultimo task più vecchio.

$$\text{reparto_scelto} = \arg \min_{r \in \text{reparti_compatibili}} \text{carico_reparto}(r) \quad (2)$$

Se tutti i reparti superano la soglia di saturazione, il task viene comunque assegnato a quello meno saturo. Con un solo reparto disponibile, l'algoritmo degrada a FIFO con priorità per massimo tempo di attesa.

Passo 7 Calcolo del ranking P Una volta assegnato il task, il ranking P ne determina l'ordine nella coda (valore più alto = elaborazione anticipata):

$$P = A \cdot t_{\text{attesa}} + B \cdot \text{priorità_asporto} + C \cdot \text{semplicità} - D \cdot \text{carico} - E \cdot \text{rischio_scorte} \quad (3)$$

Tabella 7: Termini della formula di ranking P

Termine	Significato
$A \cdot t_{\text{attesa}}$	Tempo dall'arrivo dell'ordine (urgenza crescente).
$B \cdot \text{priorità_asporto}$	Bonus per ordini asporto/urgenza con tempi vincolati.
$C \cdot \text{semplicità}$	Bonus per pietanze rapide; libera il reparto nel picco.
$-D \cdot \text{carico}$	Penalità per reparti già in stress.
$-E \cdot \text{rischio_scorte}$	Penalità per scorte prossime all'esaurimento.

Passo 8 Raggruppamento per reparto e ottimizzazione batch I task di uno stesso ordine vengono raggruppati per reparto, producendo una comanda separata per ciascuno. Pietanze identiche in arrivo entro la finestra di batch (`FINESTRA_BATCH_MS`, pari a 30 secondi nell'implementazione) vengono aggregate in un unico task cumulato.

Passo 9 Generazione comande separate Per ogni reparto coinvolto viene generata una comanda indipendente. Esempio: l'ordine #52 (birra media, costine, patatine, tiramisù) produce tre comande: BAR, GRIGLIA e CUCINA.

Passo 10 Ordinamento per priorità e invio Le comande vengono ordinate per P decrescente e inviate alla stampante del reparto. Se la stampante è offline, si usa una stampante di fallback; in assenza di fallback, la comanda appare nella dashboard admin con notifica e richiesta di conferma manuale.

Passo 11 Loop su task residui Il sistema verifica se restano task non processati. In caso affermativo il flusso ritorna al Passo 7; il ciclo termina quando tutti i task dell'ordine sono stati inviati.

Casi Limite e Comportamenti Attesi

Tabella 8: Casi limite e strategie di gestione

Caso limite	Condizione	Comportamento
Stampante offline	La stampante del reparto non risponde.	Fallback su stampante secondaria o dashboard admin con notifica.
Reparto saturo	Tutti i reparti compatibili oltre la soglia.	Assegnazione al meno saturo; degrado a FIFO con priorità attesa.
Scorta insufficiente	Quantità disponibile < richiesta.	Task bloccato; notifica alla cassa; altre righe non impattate.
Ordine misto	Pietanze per reparti diversi.	Sotto-comande indipendenti per ciascun reparto.
Picco improvviso	N ordini dello stesso piatto in breve intervallo.	Raggruppamento batch con quantità cumulata.
Reparto unico	Un solo reparto compatibile operativo.	Routing forzato; FIFO con priorità per attesa massima.

Analisi di Complessità

Siano n il numero di reparti, m il numero di task in coda, k il numero di righe valide nell'ordine.

Tabella 9: Complessità per fase

Fase	Complessità
Lettura righe ordine	$O(k)$
Verifica scorte (per task)	$O(1)$
Identificazione reparti compatibili	$O(n)$
Calcolo <code>carico_reparto</code>	$O(n \cdot m)$
Selezione <code>min(carico)</code>	$O(n)$
Calcolo ranking P	$O(1)$
Ordinamento comande	$O(k \log k)$
Invio comanda (per reparto)	$O(1)$
Totale per ordine	$O(n \cdot m + k \log k)$

In pratica $n \leq 10$ e $k \leq 20$: l'algoritmo è estremamente efficiente. Il termine dominante è $O(n \cdot m)$, dove m dipende dalla lunghezza delle code a runtime.

Pseudocodice

Le Figure 24 e 25 mostrano lo pseudocodice completo.

```
1: procedure ROUTEORDER(order)
2:   tasks  $\leftarrow []$ 
3:   for riga  $\in$  order.righe do  $\triangleright O(k)$ 
4:     task  $\leftarrow$  BUILDTASK(riga, order.timestamp, order.priorità)
5:     tasks.append(task)
6:   end for
7:   for task  $\in$  tasks do  $\triangleright O(k)$ 
8:     if  $\neg$ SCORTEDISPONIBILI(task) then
9:       BLOCCA(task); NOTIFICACASSA(task)
10:      tasks.remove(task)
11:    end if
12:  end for
13:  comande  $\leftarrow \{\}$ 
14:  for task  $\in$  tasks do  $\triangleright O(k \cdot n \cdot m)$ 
15:    reparti  $\leftarrow$  GETREPARTICOMPATIBILI(task)  $\triangleright O(n)$ 
16:     $r \leftarrow$  SELECTMINCARICO(reparti)  $\triangleright O(n \cdot m)$ 
17:    task.P  $\leftarrow$  CALCOLARANKING(task, r)  $\triangleright O(1)$ 
18:    comande[r].append(task)
19:  end for
20:  comande  $\leftarrow$  RAGGRUPPAEBATCH(comande)  $\triangleright O(k \log k)$ 
21:  for  $r \in$  KEYS(comande) do
22:    task_list  $\leftarrow$  SORTBYP(comande[r])  $\triangleright O(k \log k)$ 
23:    comanda  $\leftarrow$  GENERACOMANDA(r, task_list)
24:    INVIACOMANDA(comanda, r)
25:  end for
26: end procedure
```

Figura 24: Pseudocodice funzione principale ROUTEORDER

```

1: procedure SELECTMINCARICO(reparti)  $\triangleright O(n \cdot m)$ 
2:   best  $\leftarrow$  null; best_score  $\leftarrow +\infty$ 
3:   for  $r \in$  reparti do
4:     score  $\leftarrow \sum_{\text{task} \in r.\text{coda}} \text{task}.t_{\text{prep\_residuo}}$ 
5:     score  $\leftarrow$  score  $\times \alpha_{\text{stress}}(r) \times \beta_{\text{stampante}}(r)$ 
6:     if score < best_score then
7:       best  $\leftarrow r$ ; best_score  $\leftarrow$  score
8:     end if
9:   end for
10:  return best
11: end procedure

12: procedure CALCOLARANKING(task,  $r$ )  $\triangleright O(1)$ 
13:   $t_{\text{attesa}} \leftarrow \text{NOW} - \text{task.timestamp}$ 
14:   $\pi_a \leftarrow [\text{task.priorità} = \text{ASPORTO}] ? 1.0 : 0.0$ 
15:   $s \leftarrow 1.0 / (\text{task}.t_{\text{prep}} + 1)$ 
16:  return  $A \cdot t_{\text{attesa}} + B \cdot \pi_a + C \cdot s - D \cdot \text{carico}(r) - E \cdot \text{rischio\_scorte}(\text{task})$ 
17: end procedure

```

Figura 25: Pseudocodice funzioni ausiliarie

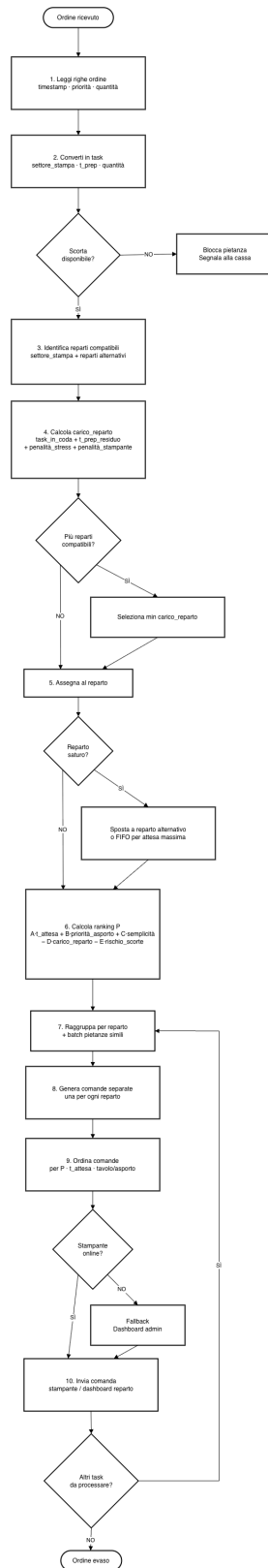


Figura 26: Diagramma di flusso Smistatore Intelligente Multi-Reparto

3.4 Test

3.4.1 Analisi statica del codice

Il backend è stato sottoposto ad analisi statica con gli stessi strumenti usati nel resto del progetto: la **complessità ciclomatica** delle funzioni è stata misurata con la regola **complexity** di **ESLint**, mentre l'**accoppiamento** tra moduli e le dipendenze circolari sono stati analizzati con **madge**. Per *isolare* il contributo dello Smistatore alla complessità del sistema, l'analisi confronta il backend *base* (privo di entrambi gli algoritmi) con lo stesso backend a cui è stato aggiunto il *solo* Smistatore, tenendo temporaneamente fuori il Predittore: la differenza tra i due stati è dunque attribuibile interamente allo Smistatore.

Tabella 10: Metriche statiche del backend: base contro stato con il solo Smistatore

Metrica (backend, src/)	Base	Con Smistatore	Δ
Complessità ciclomatica massima	51	51	=
Funzioni con complessità ciclomatica > 15	4	4	=
Funzioni totali analizzate	67	85	+18
Moduli accoppiati a <code>db.js</code> (C_a)	7	8	+1
Dipendenze circolari	0	0	=
Righe di codice (SLOC, escluse vuote e commenti)	1 506	1 732	+226

Lo Smistatore introduce 18 nuove funzioni e 226 righe di codice, senza generare dipendenze circolari. Tutte le sue funzioni restano a bassa complessità: la più impegnativa è `rigaToTasks` (15), che converte le righe d'ordine in task, mentre la funzione principale di routing `routeOrder` si ferma a 6. La complessità ciclomatica massima del sistema resta invariata (51, relativa al parser ESC/POS `parsaFlusso` nell'emulatore, estraneo allo Smistatore): l'algoritmo non peggiora cioè il punto più critico del codice, e nessuna delle funzioni introdotte supera la soglia di 15. La complessità 51 di `parsaFlusso` è un debito tecnico noto, accettato in questa fase perché il parser è stabile e isolato, e affrontato con un refactoring dedicato nell'Iterazione 3. I tre grafici seguenti visualizzano l'impatto dello Smistatore.

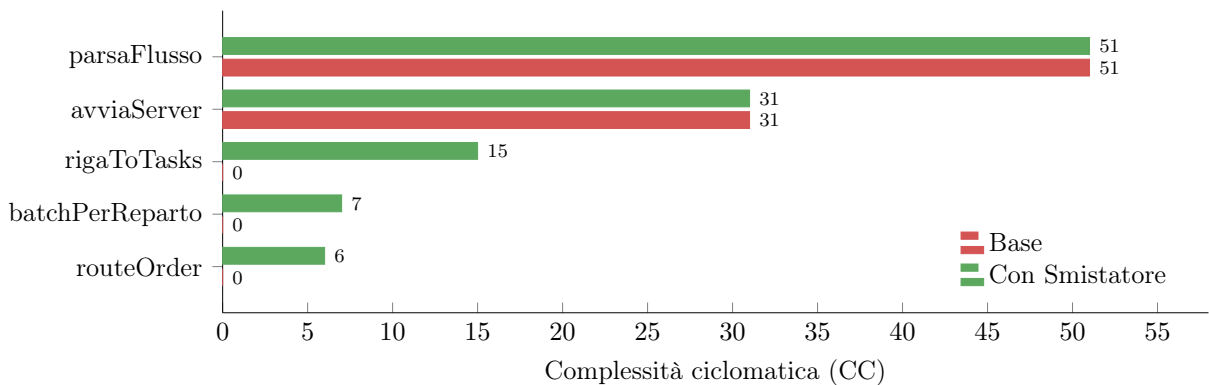


Figura 27: Complessità ciclomatica delle funzioni principali del backend. Le tre funzioni in basso (`routeOrder`, `batchPerReparto`, `rigaToTasks`) sono introdotte dallo Smistatore e restano tutte sotto la soglia di 15; le altre, legate all'infrastruttura di base, sono invariate.

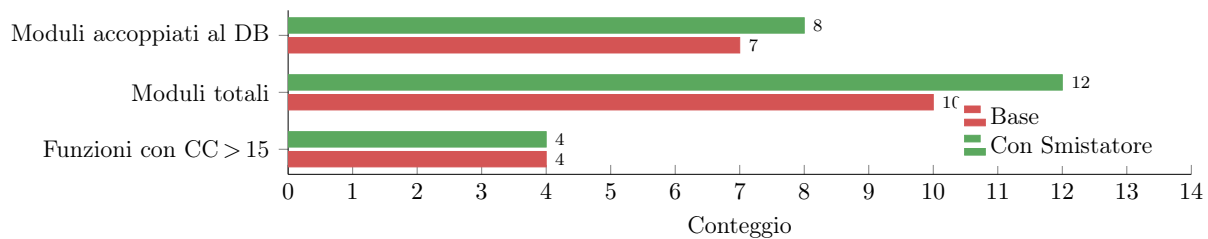


Figura 28: Metriche strutturali prima e dopo l’aggiunta del solo Smistatore: il modulo aggiunge due moduli al grafo (route e service), di cui uno (la route) accoppiato al database.

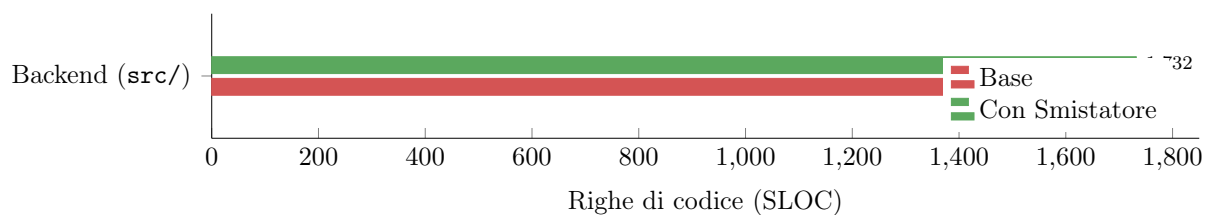


Figura 29: Dimensione del backend in righe di codice (SLOC) prima e dopo l’aggiunta del solo Smistatore.

3.4.2 Test automatizzati

Per validare la correttezza dell’algoritmo è stata sviluppata una suite di test automatizzati con **Vitest**, eseguibile con `npm test`. La suite comprende **test unitari**, che esercitano in isolamento le funzioni pure di calcolo del modulo, e **test di integrazione HTTP**, che verificano il comportamento end-to-end sulle API REST con il backend in esecuzione (saltati automaticamente quando il server non è raggiungibile).fig:test-smistatore.

```
✓ __tests__/smistatore.test.js (26 tests) 11ms
✓ Formula del carico (5)
  ✓ reparto vuoto → carico 0 1ms
  ✓ 1 task tprep=60 → carico = 60 0ms
  ✓ 5 task tprep=60 →  $\alpha=1.3$  ATTIVO → carico = 1950 0ms
  ✓ 4 task tprep=60 →  $\alpha$  NON attivo → carico =  $4 \cdot 240 = 960$  0ms
  ✓ stampante offline →  $\beta = 2.0$  0ms
✓ selectMinCarico (arg-min + tie-break) (3)
  ✓ sceglie il reparto con carico minore 0ms
  ✓ tie-break: a parità di carico, sceglie il reparto col last-task più vecchio 5ms
  ✓ tutti offline → ritorna comunque il primo candidato 0ms
✓ Routing (routeOrder) (7)
  ✓ riga con scorta sufficiente → 1 task generato 1ms
  ✓ riga con scorta INSUFFICIENTE → finisce in bloccati, NON in comande 0ms
  ✓ quantita=3 + singola_multipla → 1 task con quantita=3 0ms
  ✓ quantita=3 + singola_singola → 3 task in coda, batchati in 1 comanda qta=3 0ms
  ✓ batching: due voci diverse, stesso reparto → 1 comanda con 2 righe 0ms
  ✓ smista su più reparti e ordina le comande per priorità P decrescente 0ms
  ✓ applica i fallback ai default quando mancano settore_stampa e tempo_preparazione 0ms
✓ Ranking P (3)
  ✓ asporto incrementa P di B=50 rispetto a non-asporto 0ms
  ✓ rischio scorte (sotto soglia rosso) penalizza P di E=30 0ms
  ✓ calcolaRanking applica default se tempo_preparazione è mancante o 0 0ms
✓ Bilanciamento end-to-end con fallback bidirezionale (4)
  ✓ 6 ordini cucina con fallback → cucina_2 → distribuzione 3-3 0ms
  ✓ primario offline + fallback online → routing va al fallback 0ms
  ✓ tutti offline → routing forzato sul primario 0ms
  ✓ imposta i fallback a [] se vengono passati parametri non array 0ms
✓ completaTask (2)
  ✓ rimuove un task dalla coda del reparto 0ms
  ✓ ritorna false se il task non esiste 0ms
✓ snapshot (2)
  ✓ ritorna lo stato corrente delle code 0ms
  ✓ gestisce lo snapshot con stampante offline 0ms
```

Figura 30: Esecuzione della suite unitaria dello Smistatore con Vitest: 26/26 test unitari superati

I test unitari sono organizzati per componente dell'algoritmo:

1. **Formula del carico.** Verificano il calcolo $n \times t_{prep} \times \alpha \times \beta$, con l'attivazione del fattore di stress $\alpha = 1.3$ oltre la soglia di 5 task e del fattore $\beta = 2.0$ a stampante offline.
2. **Selezione del reparto (selectMinCarico).** Validano la scelta greedy del reparto a carico minore, il tie-break sul task più vecchio a parità di carico e il fallback quando tutte le stampanti sono offline.
3. **Routing e batching (routeOrder).** Coprono la generazione di un task per riga, il blocco delle righe a scorta insufficiente (mentre le altre proseguono), l'esplosione delle modalità *singola_singola* / *singola_multipla* e il raggruppamento di più voci dello stesso reparto in un'unica comanda.
4. **Ranking P.** Verificano il bonus di priorità per gli ordini da asporto ($B = 50$) e la penalizzazione per il rischio di esaurimento scorte ($E = 30$).
5. **Bilanciamento e fallback.** Validano la distribuzione del carico su reparti con fallback bidirezionale (6 ordini ripartiti 3-3) e il re-routing automatico verso il reparto alternativo quando la stampante primaria è offline.

3.4.3 API esposte

Gli endpoint seguenti supportano lo Smistatore e la catena di stampa; valgono le convenzioni REST definite nell'Iterazione 1. Le operazioni di scrittura (**POST**, **PUT**) sono verificate tramite Postman, di cui si riporta richiesta e risposta.

Smistatore

Esponde lo stato interno dello **Smistatore Intelligente Multi-Reparto** (UC7) e permette di aggiornare a runtime le configurazioni di fallback e lo stato delle stampanti, senza riavviare il server.

GET /api/smistatore/stato

Snapshot delle code di preparazione per ogni reparto attivo: numero di task in coda, carico stimato e stato della stampante associata.

```
{
  "cucina": { "task_in_coda": 3, "carico": 420, "stato_stampante": "online" },
  "bar":    { "task_in_coda": 1, "carico": 60, "stato_stampante": "online" },
  "griglia": { "task_in_coda": 0, "carico": 0, "stato_stampante": "offline" }
}
```

POST /api/smistatore/completa

Rimuove un task dalla coda di un reparto una volta che la preparazione è completata. Campi obbligatori: **reparto** e **voce_id**. Campo opzionale: **ordine_id** per identificare univocamente il task in caso di omonimi.

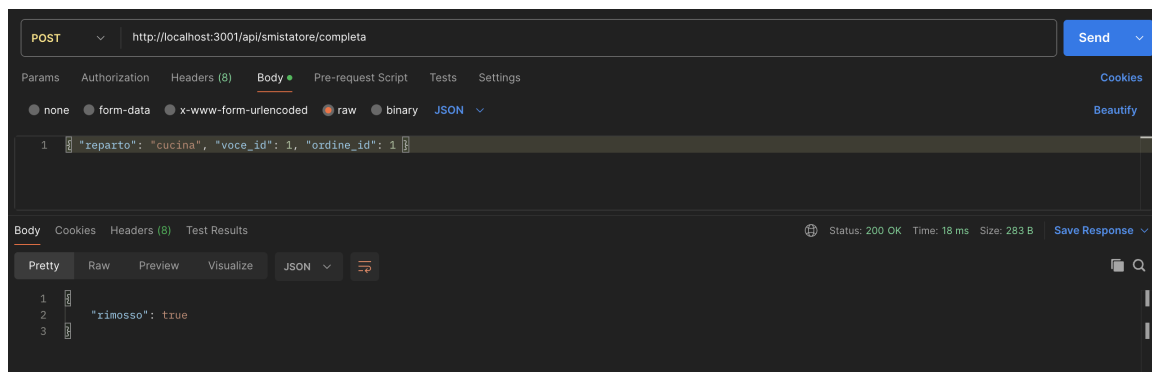


Figura 31: Postman **POST** /api/smistatore/completa

PUT /api/smistatore/stampante/:reparto

Imposta lo stato (**online** / **offline**) della stampante di un reparto, aggiornando sia la memoria interna dello smistatore che il database. Se la stampante passa a **offline**, i task successivi vengono automaticamente instradati al fallback configurato.

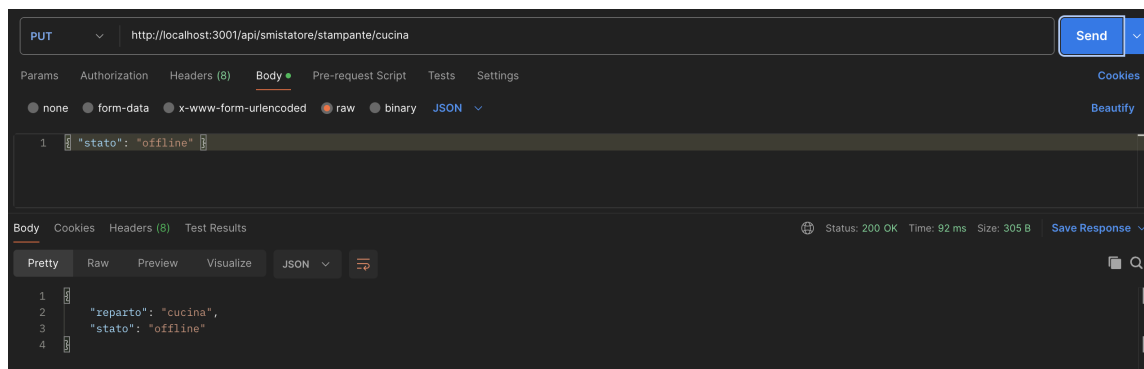


Figura 32: Postman PUT /api/smistatore/stampante/:reparto

PUT /api/smistatore/fallback/:reparto

Configura i reparti alternativi a cui instradare i task quando la stampante del reparto primario è offline. Il campo `fallback` è un array di nomi reparto.

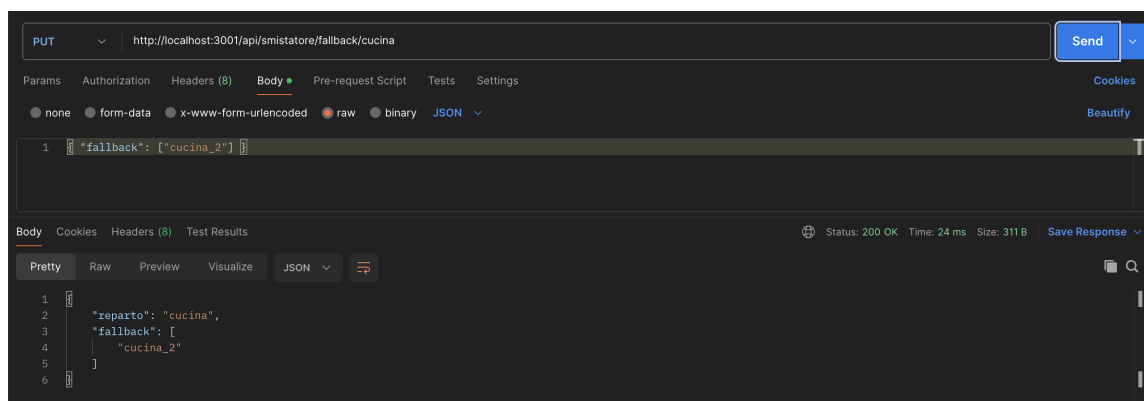


Figura 33: Postman PUT /api/smistatore/fallback/:reparto

Stampe

Gestisce lo storico delle stampe ESC/POS inviate, la ristampa di comande e il controllo dell'emulatore TCP usato in modalità demo/sviluppo.

GET /api/stampe

Storico delle stampe eseguite. Con il parametro `?ordine_id=N` restituisce solo le stampe relative a quell'ordine; senza parametro restituisce le ultime 100 in ordine cronologico decrescente.

```
[
  {
    "id": 1,
    "ordine_id": 42,
    "reparto": "cucina",
    "esito": "ok",
    "durata_ms": 85,
    "timestamp": "2026-05-26T20:14:00.000Z",
    "stampante_nome": "Cucina A",
    "indirizzo_ip": "127.0.0.1",
    "porta": 9100
  }
]
```

GET /api/stampe/recenti

Ultime `N` stampe eseguite (default 50, massimo 500) con parametro opzionale `?limit=N`. Versione alleggerita senza payload per uso nelle dashboard.

```
[
  { "id": 5, "ordine_id": 44, "reparto": "bar", "esito": "ok",
    "durata_ms": 42, "timestamp": "2026-05-26T20:15:10.000Z",
    "stampante_nome": "Stampante Bar" }
]
```

POST /api/stampe/:id/ristampa

Reinvia alla stampante la comanda corrispondente a una stampa già registrata, riutilizzando il payload JSON salvato. Risposta 400 se il payload non è disponibile; 404 se la stampa non esiste.

```
{ "ristampa": { "esito": "ok", "durata_ms": 91 } }
```

POST /api/stampe/test/:id

Invia una pagina di test alla stampante identificata da `id`. Utile per verificare la connettività prima dell'evento.

```
{ "esito": "ok", "stampante_id": 2, "durata_ms": 120 }
```

POST /api/stampe/emulatore/:stampante_id/spegni

Spegne la stampante emulata indicata, aggiorna lo stato nel database a `offline` e notifica lo smistatore tramite WebSocket (`stampante_offline`). Usato per simulare guasti in modalità demo.

```
{ "ok": true, "stampante_id": 1, "stato": "offline" }
```

POST /api/stampe/emulatore/:stampante_id/accendi

Riavvia la stampante emulata, aggiorna lo stato a `online` e notifica lo smistatore tramite WebSocket (`stampante_online`).

```
{ "ok": true, "stampante_id": 1, "stato": "online" }
```

GET /api/stampe/emulatore/stato

Snapshot dello stato di tutti gli emulatori TCP attivi (porta, stato, ultima attività).

```
{
  "1": { "porta": 9100, "stato": "online", "ultima_attivita": "2026-05-26T20:14:00Z" },
  "2": { "porta": 9101, "stato": "online", "ultima_attivita": "2026-05-26T20:12:30Z" },
  "3": { "porta": 9102, "stato": "offline", "ultima_attivita": null }
}
```

4 ITERAZIONE 3

In questa iterazione conclusiva viene sviluppato il secondo modulo algoritmico del progetto, il **Predittore Dinamico di Riordino Scorte**, vengono completate le API di gestione del menù e viene introdotta l'**autenticazione dell'amministratore**, che protegge l'accesso al backoffice. L'iterazione si chiude con il consolidamento della qualità del codice: un intervento di refactoring guidato da metriche statiche e l'analisi di copertura del backend.

4.1 Casi d'uso implementati

Tabella 11: Casi d'uso sviluppati nell'Iterazione 3

ID	Titolo	Relazione
UC8	Autenticazione Admin	definito nell'Iterazione 0
UC9	Predittore Dinamico di Riordino Scorte	realizza UC5b in forma avanzata

Analogamente a quanto fatto nell'Iterazione 2 per lo Smistatore, il modulo del Predittore riceve un identificatore dedicato (UC9) invece di riusare UC5b. Il caso d'uso *UC5b – Aggiornare Scorte*, descritto nell'Iterazione 0 e implementato nella sua forma base, viene qui realizzato in forma avanzata da UC9: per non sovraccaricare lo stesso identificatore con due significati distinti tra iterazioni, il modulo algoritmico è numerato a parte. Il caso d'uso UC8 (Autenticazione Admin) è invece definito a livello di requisiti nell'Iterazione 0 e implementato in questa iterazione.

4.2 Realizzazione dell'autenticazione Admin (UC8)

L'area di amministrazione (configurazione di menù, scorte, stampanti e parametri del Predittore) era finora raggiungibile da chiunque conoscesse l'indirizzo della pagina `/admin`: viene ora introdotto un controllo d'accesso dedicato.

Scelte realizzative

La password non viene mai memorizzata in chiaro: nel database è conservato solo il suo hash `bcrypt` (con *salt* per-hash e fattore di costo configurabile), associato all'unico utente con ruolo `admin` nella tabella `utente`, già presente nel modello dati (Figura 22). Alla verifica positiva il sistema emette un *token di sessione* opaco con scadenza, che il client presenta nelle richieste successive per accedere alle funzioni riservate; il token è volatile, quindi un riavvio del server invalida le sessioni in corso. La password di default impostata dal sistema è 1111 (configurabile via variabile d'ambiente al primo avvio) ed è modificabile dall'Admin dopo l'accesso.

4.3 Diagrammi UML

Il Predittore Dinamico di Riordino Scorte non introduce nuovi nodi o componenti nell'architettura: è realizzato come servizio interno al *Web Server* (modulo `predittore.js` in `services/`), invocato sia in modo periodico sia su evento di vendita, ed esposto verso i client tramite le API REST descritte più avanti. Per questo motivo i diagrammi UML strutturali di Deployment e Class per tipo di dato restano quelli presentati nell'Iterazione 2 (Figure 22 e 23). La logica di funzionamento del modulo è invece dettagliata dal diagramma di flusso in Figura 36.

L'**autenticazione Admin** (UC8) non incide invece sui diagrammi di *deployment* né sul modello *dati*, ma aggiunge un componente all'*Application Logic*. Sul piano dei dati si appoggia interamente alla classe **Utente** – con gli attributi `password_hash` e `ruolo` – già presente nel modello dell'Iterazione 2 (Figura 22); non introduce quindi nuove entità né nuovi nodi di *deployment*, che restano pertanto invariati. Sul piano dei *componenti*, l'autenticazione è realizzata dal componente **GestioneAutenticazione**, interno all'*Application Logic*, che espone verso *AdminFrontEnd* l'interfaccia **AutenticazioneMgmt** (operazioni di login, verifica, logout e cambio password, corrispondenti agli endpoint `/api/auth/admin/*`) e accede alla tabella `utente` attraverso il repository dedicato, in linea con la separazione dell'accesso ai dati introdotta dal refactoring di questa iterazione. La protezione delle funzioni riservate è affidata al middleware `richiediAdmin`, che valida il token di sessione e nega l'accesso con esito `401` in assenza di token o con ruolo non amministrativo. Per non rivelare l'esistenza dell'utenza, il login restituisce la medesima risposta `401` sia per password errata sia per utente assente; il token, opaco e con scadenza, è mantenuto in memoria volatile, perciò un riavvio del server invalida le sessioni in corso.

Per riflettere questa aggiunta, il Component Diagram e il Class Diagram per interfacce dell'Iterazione 2 sono aggiornati con il componente **GestioneAutenticazione** e la relativa interfaccia **AutenticazioneMgmt**, come mostrato nelle Figure 34 e 35.

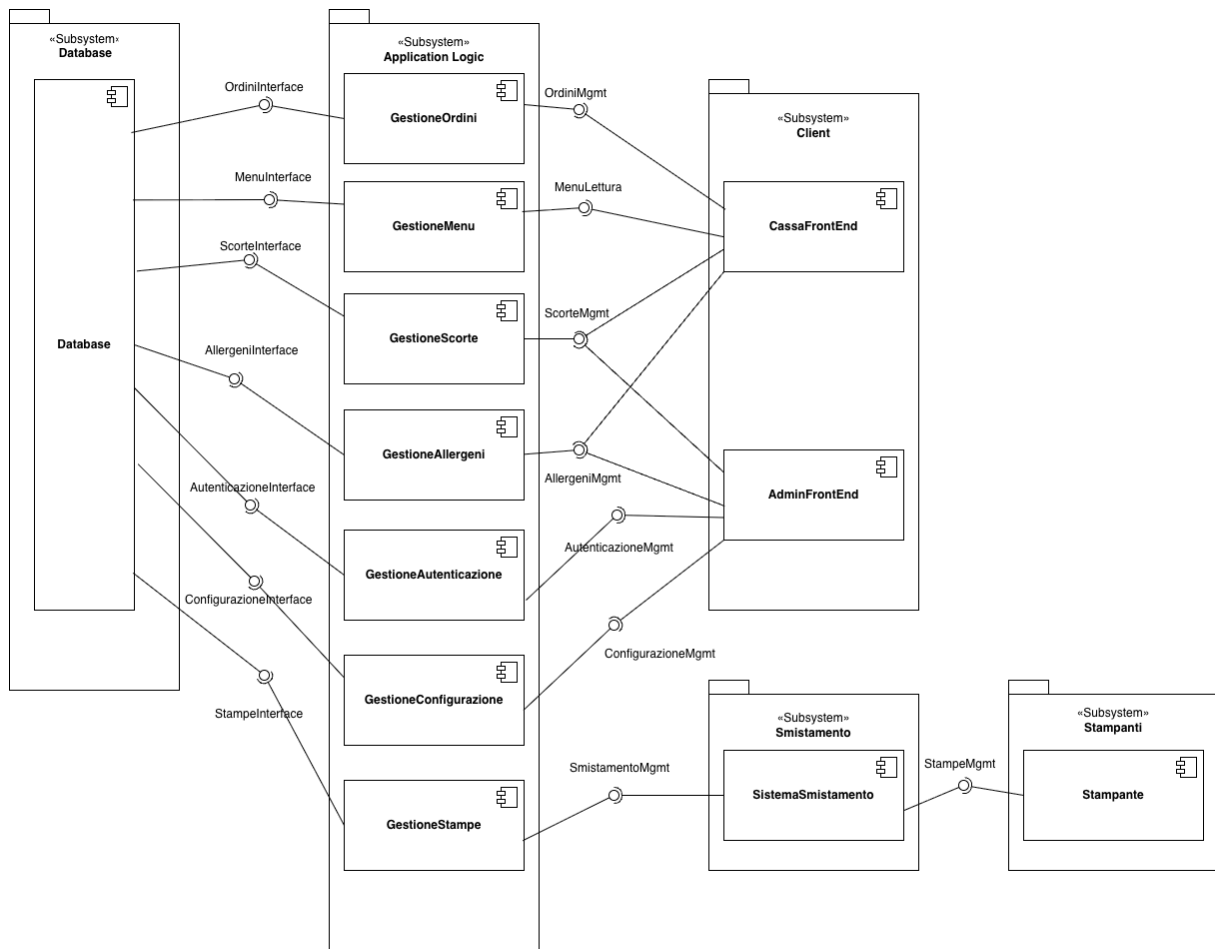


Figura 34: UML Component Diagram aggiornato con il componente **GestioneAutenticazione** e l'interfaccia **AutenticazioneMgmt** — Iterazione 3

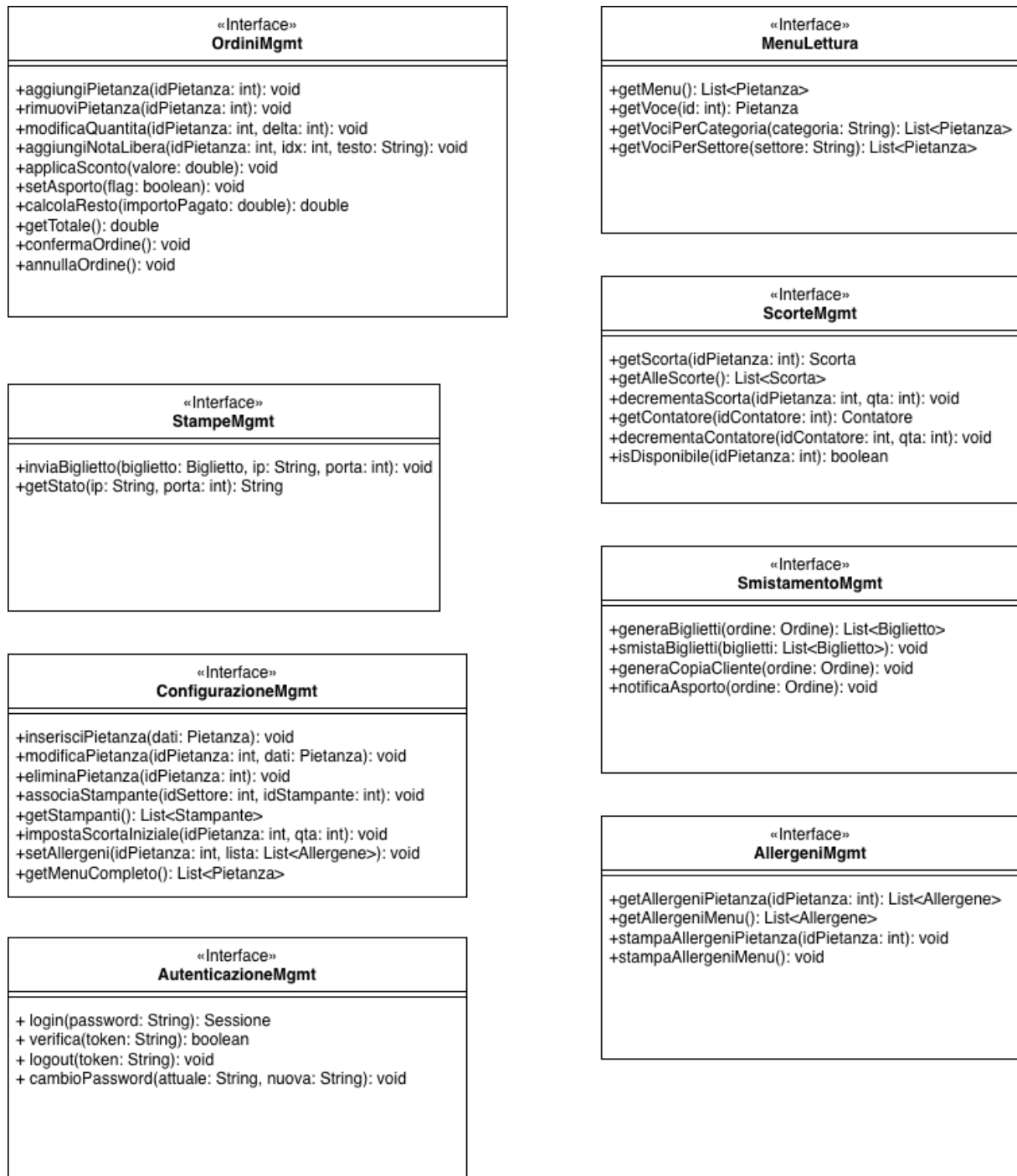


Figura 35: UML Class Diagram per interfacce aggiornato con AutenticazioneMgmt — Iterazione 3

4.4 Algoritmo: Predittore Dinamico di Riordino Scorte

Introduzione

L'algoritmo **Predittore Dinamico di Riordino Scorte** affronta la gestione reattiva delle scorte in sistemi POS per la ristorazione ad alta variabilità (sagre, eventi, locali con picchi imprevisti).

Attori coinvolti

- **Sistema POS:** emette eventi di vendita e aggiorna i contatori di consumo in tempo reale.
- **Gestore Scorte:** riceve le notifiche di rifornimento suggerito con livello di urgenza.
- **Cassa:** riceve notifiche di blocco per voci con scorta azzerata.
- **Dashboard Admin:** visualizza stato scorte, previsioni di esaurimento e azioni suggerite.

Trigger e postcondizioni

- **Trigger:** Esecuzione periodica (ogni T minuti) oppure event-driven ad ogni vendita registrata.
- **Postcondizione:** Per ogni voce attiva: stima aggiornata del tempo di esaurimento, calcolo dell'indice U , e se U supera la soglia suggerimento di rifornimento con priorità e quantità consigliata.

Parametri di Input

Ogni voce attiva viene valutata in base ai parametri della Tabella 12, provenienti dal POS, dal registro vendite e dallo storico rifornimenti.

Logica di Funzionamento

Il consumo atteso è una media pesata a tre componenti temporali comportamento recente, trend a medio termine e baseline storica:

$$\begin{aligned}\text{consumo_atteso} = & \alpha \cdot \text{consumo_ultimi_15_min} \\ & + \beta \cdot \text{consumo_ultima_ora} \\ & + \gamma \cdot \text{media_storica_stessa_fascia_oraria}\end{aligned}\tag{4}$$

con $\alpha + \beta + \gamma = 1$. In condizioni normali: $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$. In modalità picco ($\text{consumo_ultimi_15_min} > 2 \times \text{media storica}$), α sale fino a 0.8 e i pesi residui vengono ridistribuiti su β e γ .

Il tempo stimato di esaurimento:

$$\text{tempo_esaurimento} = \frac{\text{quantita_attuale}}{\text{consumo_atteso}}$$

Tabella 12: Parametri di Input dell'Algoritmo

Parametro	Descrizione
quantita_attuale	Quantità corrente in scorta, aggiornata ad ogni vendita o rifornimento.
consumo_ultimi_15_min	Unità vendute negli ultimi 15 minuti. Cattura i picchi improvvisi.
consumo_ultima_ora	Unità vendute nell'ultima ora. Trend a medio termine.
media_storica_fascia	Media storica di vendite nella stessa fascia oraria (da <i>scorta_storico</i>).
tempo_residuo_evento	Tempo alla chiusura dell'evento (secondi). Contestualizza l'urgenza.
tempo_riapprovvigionamento	Tempo stimato per completare un rifornimento (secondi).
priorita_voce	Peso nel menu: alta (principali), media, bassa (contorni/extra).
stato_scorta	Flag: disponibile / <i>soglia_giallo</i> / <i>soglia_rosso</i> / esaurito.
ingredienti_condivisi	Ingredienti comuni ad altre voci; per il consumo aggregato.
categoria_reparto	Bar, cucina, griglia. Influenza il tempo di riapprovvigionamento.

L'indice di urgenza U quantifica il rischio che la scorta si esaurisca prima del completamento di un rifornimento:

$$U = \max(0, \text{tempo_riapprovvigionamento} - \text{tempo_esaurimento}) \cdot \text{priorita_voce}$$

$U > 0$ indica che, al ritmo attuale, la scorta si esaurirà prima che il rifornimento possa arrivare. Valori elevati innescano il suggerimento di rifornimento proattivo.

Passi dell'Algoritmo

L'obiettivo è massimizzare il preavviso prima dell'esaurimento di ogni voce.

- Passo 1 Raccolta dati:** Per ogni voce attiva non marcata come “non vendibile”, il sistema legge: quantità in scorta, contatori di vendita (15 min, 1 ora), fascia oraria corrente per lo storico, metadati di reparto e priorità.
- Passo 2 Aggregazione ingredienti condivisi:** Per voci con ingredienti in comune (es. patatine condivise tra più piatti), il consumo viene aggregato su tutte le voci collegate, evitando stime divergenti.
- Passo 3 Selezione modalità di stima:** Se lo storico per la fascia oraria corrente è assente o insufficiente (< 3 occorrenze), si attiva la *modalità prudente*: stima basata solo sulle soglie statiche, con $\gamma = 0$.
- Passo 4 Rilevamento picco:** Se $\text{consumo_ultimi_15_min} > 2 \times \text{media storica}$, si attiva la *modalità picco*: α sale a 0.8 per privilegiare i dati recenti. Flag visivo in dashboard.

- Passo 5 Calcolo consumo atteso e tempo di esaurimento:** Si applica la formula ponderata con i coefficienti determinati ai passi precedenti. Se $\text{consumo_atteso} \leq 0$ (nessuna vendita nelle tre finestre), il tempo di esaurimento diventa ∞ e la voce è classificata stabile.
- Passo 6 Calcolo dell'indice U :** $U = \max(0, t_{\text{riapprov}} - t_{\text{esaurimento}}) \times \text{priorità}$. Se $t_{\text{esaurimento}} < t_{\text{residuo_evento}}$ (la voce si esaurirà prima della chiusura), U riceve un moltiplicatore di contesto aggiuntivo.
- Passo 7 Classificazione e notifica:** In base a U e allo stato della scorta (Tabella 13), il sistema classifica la voce e, se necessario, genera un suggerimento di rifornimento. Quantità suggerita: $\lceil \text{consumo_atteso} \times t_{\text{riapprov}} \times 1.3 \rceil$.
- Passo 8 Loop e aggiornamento:** Completato il ciclo su tutte le voci, il sistema attende il prossimo tick (T minuti) o la prossima vendita. Le stime vengono ricalcolate integralmente ad ogni esecuzione.

Tabella 13: Classificazione delle Voci e Azioni Conseguenti

Classificazione	Condizione	Azione
Stabile	$U = 0$ e scorta disponibile	Nessuna azione. Monitoraggio continuativo.
Attenzione	$0 < U \leq \text{soglia_warn}$	Notifica informativa in dashboard.
Urgente	$U > \text{soglia_warn}$ e scorta $\neq$ esaurita	Suggerimento di rifornimento prioritario. Alert visivo.
Critico / Esaurito	$\text{quantita_attuale} = 0$	Voce non vendibile. Notifica cassa. Richiesta conferma.

Casi Limite e Comportamenti Attesi

Analisi di Complessità

Siano n il numero di voci attive, h il numero di voci in storico per fascia oraria, s il numero medio di ingredienti condivisi per voce.

Con $n \leq 200$, $h \leq 50$, $s \leq 5$, l'algoritmo è adatto all'esecuzione in tempo reale con latenza trascurabile.

Tabella 14: Gestione dei Casi Limite

Caso limite	Condizione	Comportamento
Storico assente	Nessun dato in <code>scorta_storico</code> .	Modalità prudente: solo soglie statiche, $\gamma = 0$. Notifica admin.
Picco improvviso	<code>consumo_15min</code> $> 2 \times$ media storica.	α a 0.8. Flag picco in dashboard. Ricalcolo immediato di U .
Scorta azzerata	<code>quantita_attuale</code> = 0.	Voce non vendibile. Notifica cassa. Riattivazione solo con conferma.
Consumo nullo	Nessuna vendita nelle 3 finestre.	$t_{\text{esaurimento}} = \infty$. Voce stabile.
Ingredienti condivisi	Più voci usano lo stesso ingrediente.	Consumo aggregato. Unico indice U per l'ingrediente.
Tempo residuo $<$ riapprov.	Evento finisce prima del rifornimento.	U moltiplicato per fattore contesto. Alert immediato se urgente.
Reparto offline	Nessun aggiornamento dal reparto.	Ultimo consumo noto mantenuto. Warning staleness. Modalità prudente.

Tabella 15: Complessità Computazionale

Fase	Complessità
Lettura dati per voce	$\mathcal{O}(n)$
Aggregazione ingredienti condivisi	$\mathcal{O}(n \cdot s)$
Recupero storico fascia oraria	$\mathcal{O}(n \cdot h)$
Calcolo <code>consumo_atteso</code> , $t_{\text{esaurimento}}$, U	$\mathcal{O}(1)$ per voce
Classificazione e notifica	$\mathcal{O}(1)$ per voce
Totale per ciclo	$\mathcal{O}(n \cdot (h + s))$

Pseudocodice

Algorithm 1 Procedura principale PredictReorder

```

1: procedure PREDICTREORDER(menu_items, storico, now)  $\triangleright \mathcal{O}(n \cdot (h + s))$ 
2:   for each voce  $\in$  menu_items do  $\triangleright \mathcal{O}(n)$ 
3:     if voce.stato = 'non_vendibile' then
4:       continue
5:     end if
6:     consumo  $\leftarrow$  AGGREGACONSUMO(voce, menu_items)  $\triangleright \mathcal{O}(s)$ 
7:     if STORICODISPONIBILE(voce, now) then  $\triangleright \mathcal{O}(h)$ 
8:        $\alpha, \beta, \gamma \leftarrow$  CALCOLAPESI(voce, consumo)  $\triangleright \mathcal{O}(1)$ 
9:     else
10:       $\alpha, \beta, \gamma \leftarrow$  MODALITA_PRUDENTE  $\triangleright$  soglie statiche
11:    end if
12:     $c\_atteso \leftarrow \alpha \cdot consumo.ultimi\_15 + \beta \cdot consumo.ultima\_ora + \gamma \cdot$   

    STORICIFASCIA(voce, now)  $\triangleright \mathcal{O}(1)$ 
13:    if  $c\_atteso \leq 0$  then
14:      voce.t_esaurimento  $\leftarrow \infty$ 
15:      voce.classe  $\leftarrow$  STABILE
16:      continue
17:    end if
18:    voce.t_esaurimento  $\leftarrow voce.q\_attuale / c\_atteso$   $\triangleright \mathcal{O}(1)$ 
19:     $voce.U \leftarrow \max(0, voce.t\_riapprov - voce.t\_esaurimento) \cdot voce.priorita$   $\triangleright$   

     $\mathcal{O}(1)$ 
20:    voce.classe  $\leftarrow$  CLASSIFICA(voce)  $\triangleright \mathcal{O}(1)$ 
21:    if voce.classe  $\in$  {URGENTE, CRITICO} then
22:       $q\_sug \leftarrow \lceil c\_atteso \cdot voce.t\_riapprov \cdot SAFETY \rceil$   $\triangleright \mathcal{O}(1)$ 
23:      GENERANOTIFICA(voce,  $q\_sug$ )  $\triangleright \mathcal{O}(1)$ 
24:    end if
25:  end for
26: end procedure

```

Algorithm 2 Funzioni ausiliarie

```
1: procedure AGGREGACONSUMO(voce, menu_items) ▷  $\mathcal{O}(s)$ 
2:    $c \leftarrow voce.consumo\_raw$ 
3:   for each ing ∈ voce.ingredienti_condivisi do
4:     for each altra ∈ menu_items do
5:       if ing ∈ altra.ingredienti then
6:          $c \leftarrow c + altra.consumo\_raw$ 
7:       end if
8:     end for
9:   end for
10:  return  $c$ 
11: end procedure

12: procedure CALCOLAPESI(voce, consumo) ▷  $\mathcal{O}(1)$ 
13:  if consumo.ultimi_15 >  $2.0 \times \text{STORICIFASCIA}(voce, now)$  then
14:    return  $\alpha = 0.8, \beta = 0.15, \gamma = 0.05$  ▷ modalità piccolo
15:  else
16:    return  $\alpha = 0.5, \beta = 0.30, \gamma = 0.20$  ▷ modalità normale
17:  end if
18: end procedure

19: procedure CLASSIFICA(voce) ▷  $\mathcal{O}(1)$ 
20:  if voce.q_attuale = 0 then return CRITICO
21:  end if
22:  if voce.U = 0 then return STABILE
23:  end if
24:  if voce.U ≤ SOGLIA_WARN then return ATTENZIONE
25:  end if
26:  return URGENTE
27: end procedure
```

Note Implementative

- **Configurazione:** α , β , γ , SOGLIA_WARN e SAFETY sono configurabili per evento/locale via pannello admin. Default: SOGLIA_WARN = 300s, SAFETY = 1.3, tick $T = 5$ min.
- **Persistenza:** i contatori a 15 min e 1 ora sono in memoria volatile ad alta frequenza. Lo storico multi-evento viene scritto su *scorta_storico* a fine evento per alimentare le previsioni future.
- **Integrazione con UC7:** il Predittore può essere invocato dallo Smistatore Intelligente prima del routing, bloccando in anticipo l'assegnazione di task per voci CRITICO ed evitando comande non evadibili.

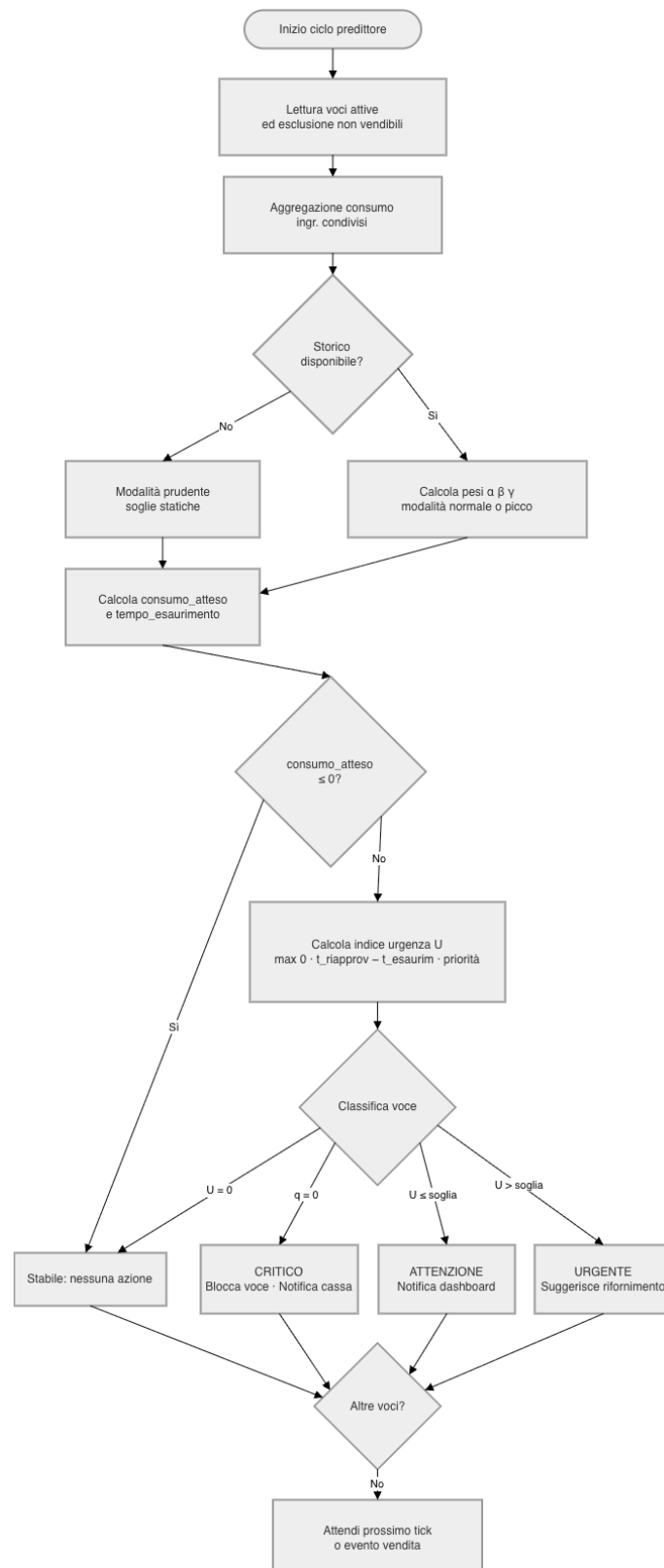


Figura 36: Diagramma di flusso Predittore Dinamico di Riordino Scorte

4.5 Test

4.5.1 Analisi statica del codice e refactoring

Il codice è stato sottoposto a un intervento di refactoring guidato da metriche statiche: la complessità ciclomatica delle funzioni (misurata con ESLint), l'accoppiamento tra moduli e le dipendenze circolari (analizzati con madge) e la dimensione dei moduli in righe di codice. L'obiettivo non era aggiungere funzionalità, ma rendere il codice più semplice da leggere e da mantenere riducendo la complessità delle funzioni più critiche e l'accoppiamento verso il database.

Interventi sul backend

Separazione dell'accesso ai dati. Nella versione iniziale le query SQL erano scritte direttamente all'interno dei gestori delle route, mescolando in uno stesso file la logica HTTP, l'accesso al database e la costruzione delle risposte. Le query sono state spostate in un layer dedicato (la cartella `repositories/`), con un unico punto di accesso al pool di connessioni. In questo modo le route delegano la persistenza ai repository e non conoscono più il dettaglio delle tabelle. Il numero di moduli che dipendono direttamente da `db.js` passa da 10 a 2 (si veda la Figura 38), e la stessa estrazione ha permesso di alleggerire i gestori che prima contenevano le query.

Sottosistema di stampa dietro una *façade*. La catena che porta dall'ordine confermato alla stampa (smistatore, formatter, dispatcher ed emulatore ESC/POS) era invocata direttamente dalle route, che quindi dipendevano da quattro servizi distinti. È stata incapsulata dietro una *façade* (`services/stampa/`) che espone poche operazioni di alto livello: le route ora dipendono da un solo modulo. Contestualmente il dispatcher e l'emulatore non accedono più al database in modo diretto ma tramite i repository, riducendo ulteriormente l'accoppiamento.

Riduzione della complessità. Le due funzioni più complesse del backend sono state ristrutturare. Il parser del flusso ESC/POS (`parseFlusso`), costituito da un unico ciclo con una lunga catena di condizioni sui codici di comando, è stato riscritto con una tabella di dispatch che associa a ogni comando una piccola funzione dedicata: la complessità ciclomatica scende da 51 a 11. La funzione di predizione delle scorte (`eseguiPredizione`) è stata scomposta in funzioni pure più piccole (la valutazione della singola voce e la derivazione dello stato di scorta), passando da 21 a 2.

Interventi sul frontend

I due componenti principali dell'interfaccia, `Admin.jsx` e `Cassa.jsx`, concentravano in un unico file lo stato, la logica, le chiamate al backend e numerose finestre modali: rispettivamente 1489 e 742 righe. Sono stati scomposti in circa venti tra componenti e hook a responsabilità singola, introducendo un client centralizzato per le chiamate API ed estraendo in funzioni separate la validazione dei dati e la costruzione dei payload. La dimensione di `Admin.jsx` scende da 1328 a 150 righe di codice e quella di `Cassa.jsx` da 669 a 227 (Figura 39).

Risultati

I tre grafici seguenti confrontano le metriche prima e dopo il refactoring.

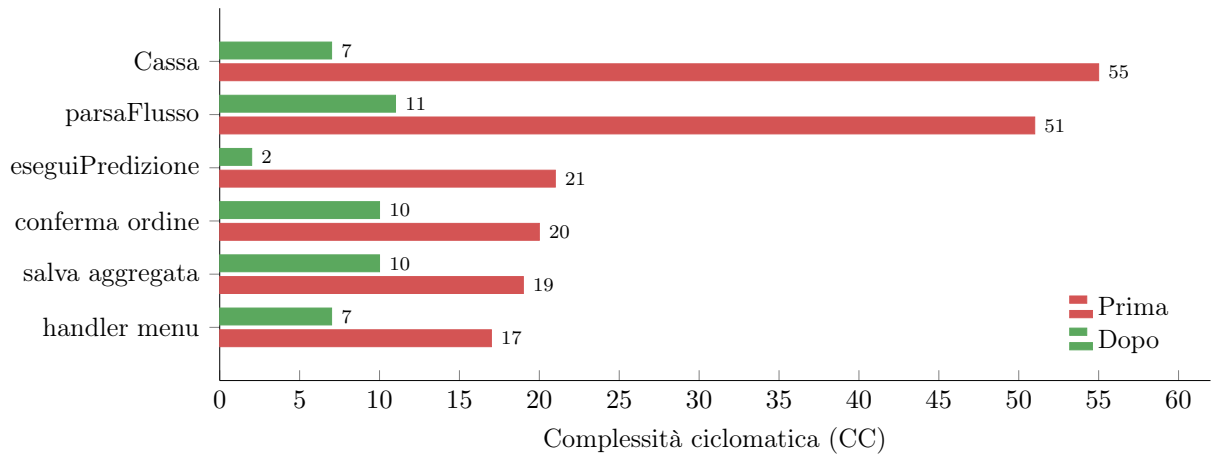


Figura 37: Complessità ciclomatica delle funzioni più critiche

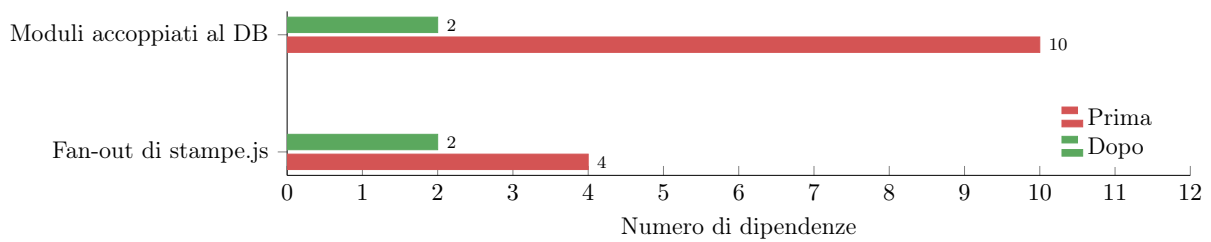


Figura 38: Accoppiamento prima e dopo.

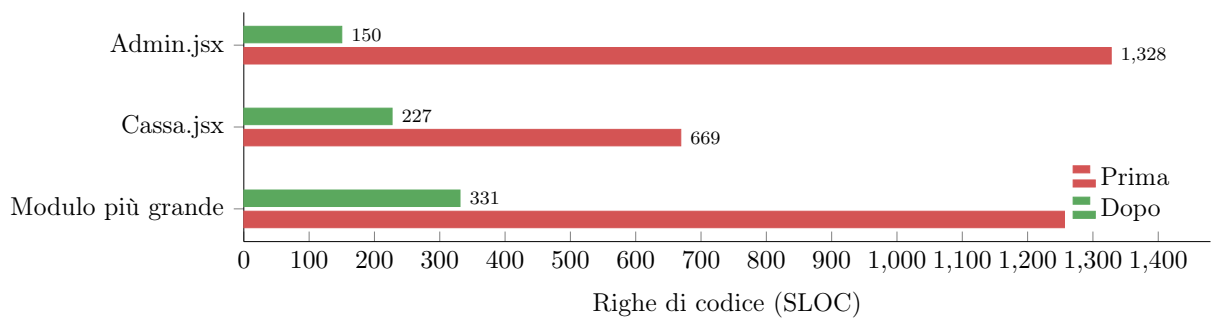


Figura 39: Coesione vista come decomposizione dei moduli: i due “God Component” del frontend e il modulo più grande del progetto vengono divisi in unità a responsabilità singola.

Metrica	Prima	Dopo
Funzioni con complessità ciclomatica > 15	11	5
Complessità ciclomatica massima	55	37
Moduli accoppiati al DB (C_a di <code>db.js</code>)	10	2
Dipendenze circolari	0	0
Modulo più grande (righe di codice)	1328	331
Numero di moduli	19	50

Tabella 16: Sintesi delle metriche prima e dopo il refactoring.

4.5.2 Test automatizzati

Come per lo Smistatore, il Predittore è coperto da una suite di test automatizzati con Vitest

```
✓ __tests__/predittore.test.js (33 tests) 6ms
✓ calcolaPesi: selezione modalità (4)
  ✓ storico sufficiente + consumo normale → modalità "normale" ( $\alpha=0.5$ ) 1ms
  ✓ storico sufficiente + consumo a picco ( $>2\times$  storico) → modalità "picco" ( $\alpha=0.8$ ) 0ms
  ✓ occorrenze storiche  $< 3$  → modalità "prudente" ( $\gamma=0$ ) 0ms
  ✓ storico = 0 → niente picco anche con consumo alto (no divisione su 0) 0ms
✓ consumoAtteso: applica formula (2)
  ✓  $\alpha \cdot c15/15 + \beta \cdot c1h/60 + \gamma \cdot storico/60$  0ms
  ✓ zero consumo ovunque → 0 0ms
✓ tempoEsaurimentoSec (3)
  ✓ quantità=10 + consumo=1 unità/min → 600 secondi 0ms
  ✓ consumo  $\leq 0$  → Infinity 0ms
  ✓ quantità=0 → 0 secondi 0ms
✓ calcolaU: indice di urgenza (5)
  ✓ tempo_esaur > tempo_riapp → U = 0 (nessuna urgenza) 0ms
  ✓ tempo_esaur < tempo_riapp con priorità media → U = (riapp - esaur)  $\times 1.0 \times 1.2$  0ms
  ✓ priorità alta moltiplica U per 1.5 0ms
  ✓ priorità bassa moltiplica U per 0.5 0ms
  ✓ moltiplicatore contesto NON attivo se esaur > residuo evento 0ms
✓ classifica: 4 stati (4)
  ✓ quantità = 0 → "esaurito" 0ms
  ✓ U = 0 + scorta disponibile → "stabile" 0ms
  ✓ U > 0 ma  $\leq$  soglia_warn → "attenzione" 0ms
  ✓ U > soglia_warn → "urgente" 0ms
✓ quantitàSuggestita (2)
  ✓ consumo=0.5 unità/min, t_riapp=600s (10min) →  $\text{ceil}(0.5 \times 10 \times 1.3) = 7$  0ms
  ✓ consumo=2 unità/min, t_riapp=900s (15min) →  $\text{ceil}(2 \times 15 \times 1.3) = 39$  0ms
✓ aggregaConsumiCondivisi (1)
  ✓ voci [1,2] condivise → entrambe vedono la somma dei consumi 0ms
✓ tempoResiduoEventoSec (2)
  ✓ orario futuro nello stesso giorno → secondi positivi 0ms
  ✓ orario passato → 0 0ms
✓ derivaStatoVisivo (5)
  ✓ scorta non attiva → "sconosciuto" 0ms
  ✓ quantità = 0 → "esaurito" 0ms
  ✓ quantità  $\leq$  soglia_rosso → "critico" 0ms
  ✓ quantità  $\leq$  soglia_giallo → "attenzione" 0ms
  ✓ quantità sopra tutte le soglie → "disponibile" 0ms
✓ valutaVoce: pipeline completa (2)
  ✓ voce senza consumo → stabile con tempo_esaurimento null 0ms
  ✓ voce con quantità=0 → esaurito 0ms
✓ eseguiPredizione (mock database) (3)
  ✓ calcola la predizione aggregando i dati dal mock del repository 1ms
  ✓ calcola la predizione per una singola voce (eseguiPredizionePerVoce) 1ms
  ✓ gestisce indici condivisi (tabella voce_contatore) e catch errori 1ms
```

Figura 40: Esecuzione della suite unitaria del Predittore con Vitest: 33/33 test unitari superati.

I test unitari sono organizzati per funzione interna dell'algoritmo:

1. **Selezione della modalità** (calcolaPesi). Verificano la scelta tra le modalità *normale*, *picco* e *prudente* con i pesi α , β , γ che sommano sempre a 1 e la corretta gestione del caso di storico nullo, senza divisioni per zero.
2. **Consumo atteso e tempo di esaurimento** (consumoAtteso, tempoEsaurimentoSec). Validano la media pesata a tre componenti temporali e il calcolo del tempo di esaurimento, che con consumo nullo o negativo deve restituire ∞ .

3. **Indice di urgenza U (calcolaU).** Verificano l'effetto dei moltiplicatori di priorità (alta 1.5, media 1.0, bassa 0.5) e del moltiplicatore di contesto evento (1.2), oltre al caso $U = 0$ quando la scorta si esaurisce dopo il completamento del rifornimento.
4. **Classificazione (classifica).** Validano l'assegnazione dei quattro stati *esaurito*, *stabile*, *attenzione* e *urgente* in base a U e allo stato della scorta.
5. **Quantità suggerita e aggregazioni** (quantitaSuggestita, aggregaConsumiCondivisi, tempoResiduoEventoSec). Coprono la formula $\lceil \text{consumo} \cdot t_{riapp} \cdot 1.3 \rceil$, il consumo aggregato tra voci con ingredienti condivisi e il calcolo del tempo residuo dell'evento.

Test autenticazione Admin

L'autenticazione Admin (UC8) è coperta da una suite **Vitest** dedicata, articolata come per gli altri moduli in **test unitari** sulle funzioni pure del servizio di autenticazione e **test di integrazione HTTP** sugli endpoint. L'esito complessivo della suite è riportato in Figura 41.

I **test unitari** (file `__tests__/auth.test.js`) verificano in isolamento dal database:

1. **Verifica password** (hashPassword, verificaPassword): l'hash prodotto è in formato `bcrypt` e non in chiaro; la verifica restituisce *vero* solo con la password corretta e *falso* (senza sollevare eccezioni) con password errata o hash assente/malformato.
2. **Ciclo di vita del token** (creaToken, validaToken, revocaToken): emissione di un token con scadenza futura, riconoscimento di un token valido, rifiuto di token inesistenti, scaduti (con rimozione automatica) o revocati dopo il logout.
3. **Estrazione del token** (estraiToken): lettura del token dall'header `Authorization: Bearer` e gestione dei casi di header assente o con formato errato.
4. **Middleware di protezione** (richiediAdmin): accesso consentito con token admin valido, negato con esito 401 in assenza di token o con token di ruolo non amministrativo.

I **test di integrazione HTTP** (file `__tests__/auth-integrazione.test.js`) validano end-to-end il flusso sugli endpoint reali: login con password corretta (200 con token), login con password errata (401) o assente (400), verifica del token, protezione delle operazioni riservate in assenza di token (401), logout con invalidazione del token e cambio password con successivo ripristino del valore di default per garantire l'idempotenza tra esecuzioni.

```

✓ __tests__/auth.test.js (15 tests) 510ms
✓ auth.service - password (5)
  ✓ hashPassword produce un hash bcrypt verificabile 68ms
  ✓ verificaPassword ritorna true con la password corretta 130ms
  ✓ verificaPassword ritorna false con password errata 184ms
  ✓ verificaPassword ritorna false (senza lanciare) con hash assente o malformato 0ms
  ✓ accetta la password anche se passata come numero 123ms
✓ auth.service - token di sessione (5)
  ✓ creaToken restituisce un token e una scadenza futura 1ms
  ✓ validaToken riconosce un token valido e ne legge il ruolo 0ms
  ✓ validaToken ritorna null per token inesistente o assente 0ms
  ✓ validaToken ritorna null per token scaduto e lo rimuove 0ms
  ✓ revocaToken invalida il token (logout) 0ms
✓ auth.service - estraiToken (2)
  ✓ estrae il token dall'header Authorization Bearer 0ms
  ✓ ritorna null in assenza di header o con formato errato 0ms
✓ auth.service - middleware richiediAdmin (3)
  ✓ chiama next() e popola req.utente con un token admin valido 0ms
  ✓ risponde 401 senza token 0ms
  ✓ risponde 401 con un token di ruolo non admin 0ms
✓ __tests__/auth-integrazione.test.js (8 tests) 843ms

```

Figura 41: Esecuzione della suite di autenticazione Admin con Vitest

4.5.3 Analisi di copertura del backend

Validazione con Vitest

La copertura del codice è misurata con il provider v8 di Vitest, configurato per analizzare i moduli algoritmici in `services/` (`predittore.js` e `smistatore.js`). I test che la esercitano comprendono sia i test unitari sia i test di integrazione HTTP eseguiti in ambiente Docker, già descritti nelle sezioni *Test automatizzati* delle Iterazioni 2 e 3.

Struttura della suite. Le suite dei due moduli algoritmici comprendono complessivamente **71 test** distribuiti su 4 file (di cui **59 test unitari** e **12 test di integrazione**); a queste si aggiungono i **23 test** dell'autenticazione Admin (si veda la sezione *Test dell'autenticazione Admin*), per un totale di **94 test** su 6 file. Le metriche di copertura riportate di seguito si riferiscono specificamente ai due moduli algoritmici `predittore.js` e `smistatore.js`, sui quali è configurato il provider di copertura.

Setup dei test di integrazione. Ogni test di integrazione verifica un flusso end-to-end reale: la richiesta HTTP attraversa il router Express, attiva la logica di business (`smistatore` o `predittore`), interagisce con il database MySQL/MariaDB e, ove previsto, pilota gli emulatori di stampa.

Per garantire l'*idempotenza* tra esecuzioni successive, ogni test è preceduto da una fase di `beforeEach` che:

1. riaccende tutte le stampanti emulate (`POST /api/stampe/emulatore/:id/accendi`);
2. ripristina le scorte delle voci utilizzate nei test a quantità elevate tramite `PUT /api/scorte/:voce_id`;
3. drena le code interne dello smistatore (`POST /api/smistatore/completa`) per annullare il carico residuo di esecuzioni precedenti.

Tabella 17: Test di integrazione HTTP eseguiti con backend Docker attivo.

Modulo	Caso di test	Verifica
smistatore	GET /api/stampanti restituisce almeno 4 stampanti	Configurazione DB
	Ordine multi-reparto → comande corrette + audit OK	Routing + stampa
	Stampante spenta + ordine cucina → routing su cucina_2	Fallback dinamico
	Ordine asporto → $P_{\text{asporto}} > P_{\text{tavolo}}$ con $\Delta P \geq 40$	Bonus priorità B
	Scorta esaurita → riga in bloccati , le altre proseguono	Guardia scorte
predittore	GET /api/predittore/scorte ritorna lista voci con tutti i campi	Schema risposta
	Filtro ?stato=stabile restituisce solo voci stabili	Query filtering
	GET /api/predittore/configurazione ritorna i 3 parametri	Lettura config
	PUT .../configurazione/:k aggiorna e persiste il valore	Scrittura config
	Voce con quantità = 0 → stato "esaurito"	Classificazione
	Voce senza consumo recente → tempo_esaurimento = null	Caso limite
	Conteggi coerenti con il totale delle predizioni	Integrità dati

```

RUN v3.2.6 C:/Users/masig/Documents/GitHub/Progetto-PAC/CODE/gestionale-feste/backend
Coverage enabled with v8

_ _tests_/predittore.test.js (33 tests) 50ms
_ _tests_/predittore-integrazione.test.js (7 tests) 310ms
_ _tests_/smistatore.test.js (26 tests) 40ms
_ _tests_/smistatore-integrazione.test.js (5 tests) 4023ms
  [ ] Integrazione HTTP smistatore > GET /api/stampanti restituisce almeno 4 stampanti 920ms
  [ ] Integrazione HTTP smistatore > Ordine multi-reparto → comande corrette + audit OK 1355ms
  [ ] Integrazione HTTP smistatore > Stampante spenta + ordine cucina → routing automatico su cucina_2 1089ms
  [ ] Integrazione HTTP smistatore > Ordine con asporto → comanda con P più alto rispetto a ordine tavolo 323ms
  [ ] Integrazione HTTP smistatore > Scorta esaurita → riga in bloccati, le altre proseguono 332ms

Test Files 4 passed (4)
Tests 71 passed (71)
Start at 14:56:15
Duration 5.00s (transform 173ms, setup 0ms, collect 780ms, tests 4.42s, environment 2ms, prepare 1.20s)

% Coverage report from v8
-----
File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files     | 100     | 94.4     | 100     | 100     |
predittore.js | 100     | 93.75    | 100     | 100     | 43,70-71,236,269
smistatore.js | 100     | 95.06    | 100     | 100     | 34,148,176-177
-----

```

Figura 42: Esecuzione completa della suite Vitest con Docker attivo e report di copertura: 71/71 test superati (4 file, nessuno saltato), copertura globale al 94,4% di branch (100% su statement, funzioni e righe).

Casi di test di integrazione. La copertura globale si attesta al 94,4% di branch perché il provider v8 di Vitest è configurato per misurare i soli moduli in **services/**

(`predittore.js` e `smistatore.js`): i rami aggiuntivi esercitati dai test di integrazione ricadono infatti in `routes/` e `repositories/`, che si trovano al di fuori del perimetro misurato.

Questi test restano comunque essenziali, perché verificano l'intera catena *HTTP* → *router* → *service* → *repository* → *database* → *emulatore di stampa* in condizioni realistiche (errori di rete, latenza TCP, stato persistente, concorrenza tra stampanti) non raggiungibili o verificabili dai soli test unitari.

4.5.4 API esposte

Gli endpoint seguenti supportano il Predittore, realizzano l'autenticazione Admin e completano la gestione del menù; valgono le convenzioni REST definite nell'Iterazione 1. Le operazioni di scrittura (**POST**, **PUT**) sono verificate tramite Postman, di cui si riporta richiesta e risposta.

Predittore

Espone le previsioni di esaurimento scorte calcolate dal **Predittore Dinamico di Riordino Scorte** (UC9) e permette di configurarne i parametri a runtime.

GET /api/predittore/scorte

Esegue il ciclo di predizione su tutte le voci attive e restituisce le previsioni ordinate per indice di urgenza *U* decrescente. Supporta due parametri opzionali di filtro:

- `?stato=urgente,esaurito` filtra per stato (CSV);
- `?reparto=cucina` filtra per reparto di stampa.

```
{
  "generato_a": "2026-05-26T20:20:00.000Z",
  "configurazione": { "evento_fine_oggi": "23:30",
                      "soglia_warn_urgenza": "300",
                      "giorni_storico": "30" },
  "conteggi": { "totale": 12, "stabile": 8,
                "attenzione": 2, "urgente": 1, "esaurito": 1 },
  "predizioni": [
    { "voce_id": 2, "nome": "Polenta taragna", "stato": "urgente",
      "U": 520.0, "quantita_attuale": 4,
      "tempo_esaurimento_min": 18, "quantita_suggerita": 30,
      "reparto": "cucina" }
  ]
}
```

GET /api/predittore/scorte/:voce_id

Predizione dettagliata per una singola voce. Risposta 404 se la voce non esiste o non ha la gestione scorte attiva.

```
{ "voce_id": 2, "nome": "Polenta taragna", "stato": "urgente",
  "U": 520.0, "quantita_attuale": 4,
  "consumo_atteso": 2.1, "tempo_esaurimento_min": 18,
```

```
"quantita_suggerita": 30, "reparto": "cucina",  
"modalita": "normale" }
```

GET /api/predittore/configurazione

Restituisce la configurazione corrente del predittore dalla tabella `configurazione`.

```
{  
  "evento_fine_oggi": "23:30",  
  "soglia_warn_urgenza": "300",  
  "giorni_storico": "30"  
}
```

PUT /api/predittore/configurazione/:chiave

Aggiorna o crea (upsert) un singolo parametro di configurazione del predittore. Campo obbligatorio: `valore`.

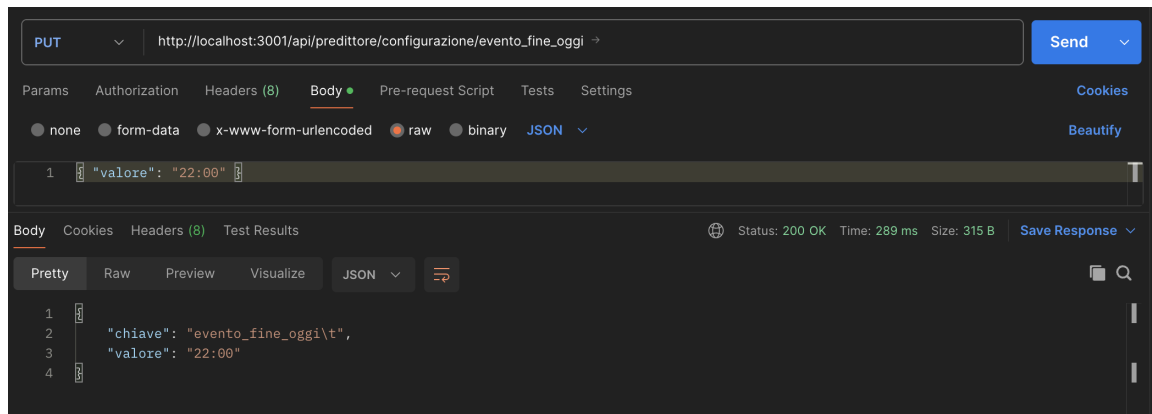


Figura 43: Postman — PUT /api/predittore/configurazione/:chiave

PUT /api/predittore/voce/:id

Aggiorna i parametri di previsione specifici di una voce: tempo di riapprovvigionamento (secondi) e/o priorità (bassa, media, alta). Almeno un campo deve essere presente.

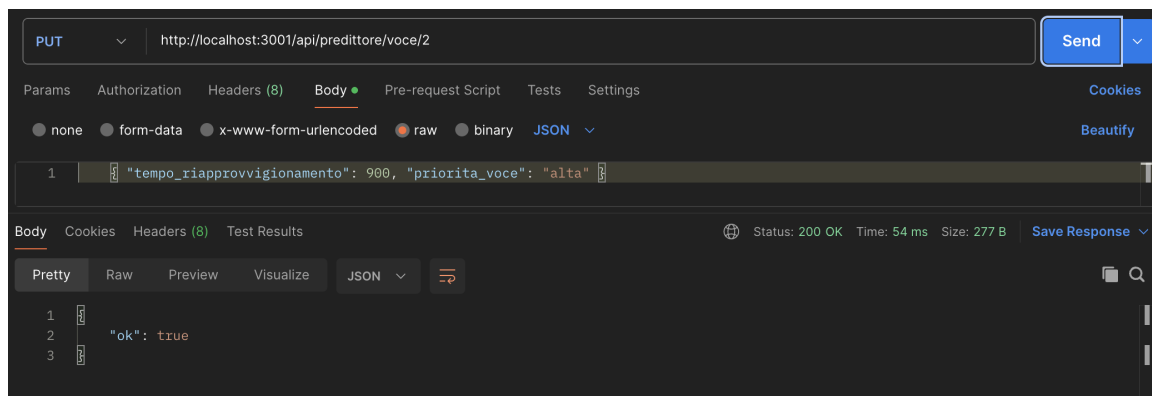


Figura 44: Postman — PUT /api/predittore/voce/:id

Menu endpoint aggiuntivi

Completano la documentazione API del Menu dell'Iterazione 1 con le funzionalità di gestione settori, pietanze aggregate, allergeni e opzioni di configurazione, a supporto dei componenti `GestioneAllergeni` e `GestioneConfigurazione` introdotti nell'architettura dell'Iterazione 2.

GET /api/menu/settori

Restituisce la lista distinta dei settori di visualizzazione con il conteggio delle voci e il colore dominante del settore.

```
[
  { "settore_visualizzazione": "Bar",    "num_pietanze": 6, "colore": "#2980B9" },
  { "settore_visualizzazione": "Primi",  "num_pietanze": 6, "colore": "#E8A838" }
]
```

GET /api/menu/opzioni

Restituisce i valori distinti dei campi usati nei dropdown del pannello Admin (categorie, settori di stampa, settori di visualizzazione).

```
{
  "categorie":          ["Bevanda", "Contorno", "Dolce", "Primo", "Secondo"],
  "settori_stampa":     ["bar", "cucina", "griglia"],
  "settori_visualizzazione": ["Bar", "Contorni", "Dolci", "Primi", "Secondi"]
}
```

GET /api/menu/tutti-allergeni

Restituisce una mappa `voce_id` → lista allergeni per *tutte* le voci del menu in un'unica chiamata. Usato dalla cassa per il precarico all'avvio, evitando richieste individuali per ogni prodotto.

```
{
  "3": [{ "id": 1, "nome": "Glutine", "descr": "Cereali contenenti glutine" },
        { "id": 4, "nome": "Uova",    "descr": "Uova e derivati" }],
  "16": [{ "id": 5, "nome": "Solfiti", "descr": "Anidride solforosa e solfiti"}]
}
```

GET /api/menu/:id/allergeni

Allergeni associati a una singola voce. Risposta 200 con array vuoto se nessun allergene è registrato per quella voce.

```
[
  { "id": 1, "nome": "Glutine", "descr": "Cereali contenenti glutine" },
  { "id": 4, "nome": "Uova",    "descr": "Uova e derivati" }
]
```

GET /api/menu/:id/composizione

Restituisce i componenti di una pietanza aggregata (es. menu fisso composto da più voci singole). Array vuoto se la voce non è aggregata.

```
[
  { "id": 1, "codice": "P001", "nome": "Risotto ai funghi",
    "prezzo": "9.00", "categoria": "Primo",
    "settore_visualizzazione": "Primi" },
]
```



```
{ "id": 7, "codice": "S001", "nome": "Costine alla griglia",
  "prezzo": "12.00", "categoria": "Secondo",
  "settore_visualizzazione": "Secondi" }
]
```

POST /api/menu/aggregata

Crea una pietanza aggregata composta da più voci esistenti. Il codice è auto-generato come per POST /api/menu. Campi obbligatori: **nome** e **componenti** (array di id di almeno 2 voci). Crea automaticamente una scorta di default.

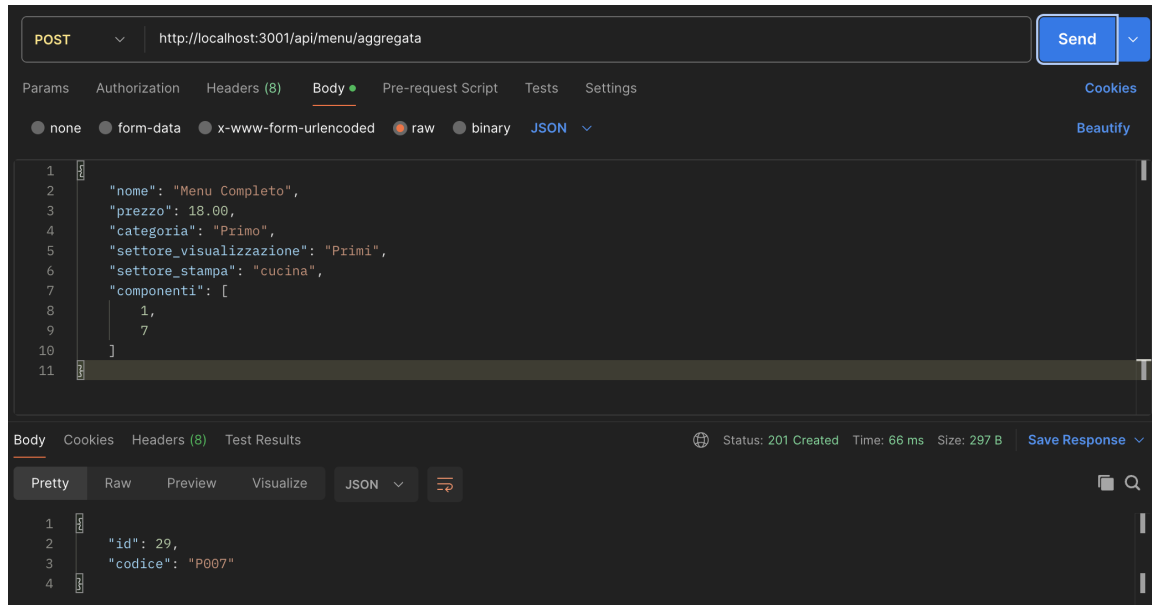


Figura 45: Postman — POST /api/menu/aggregata

PUT /api/menu/settori/rinomina

Rinomina un settore di visualizzazione aggiornando **settore_visualizzazione** su tutte le voci che vi appartengono. Campi obbligatori: **vecchio_nome** e **nuovo_nome**.

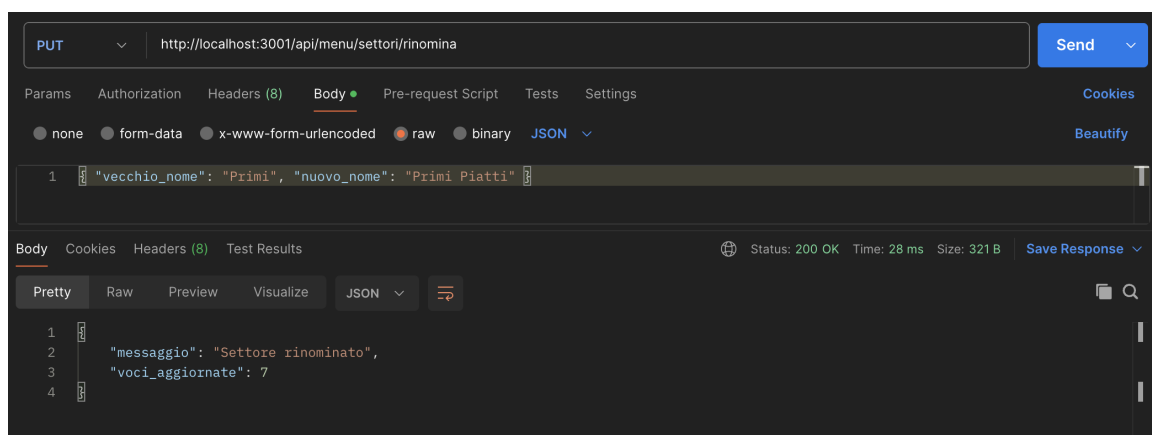


Figura 46: Postman — PUT /api/menu/settori/rinomina

PUT /api/menu/:id/composizione

Aggiorna i componenti di una pietanza aggregata esistente, sostituendo interamente le associazioni precedenti. Richiede `componenti` (array di almeno 2 id).

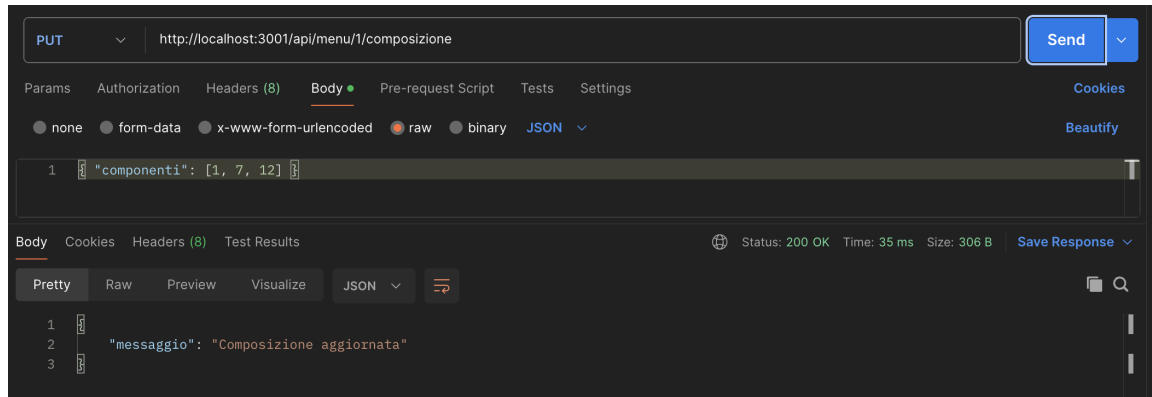


Figura 47: Postman — PUT /api/menu/:id/composizione

DELETE /api/menu/settori/:nome

Elimina un intero settore e tutte le voci che vi appartengono (CASCADE sul database). Operazione irreversibile.

```
{ "messaggio": "Settore eliminato", "voci_eliminate": 6 }
```

Autenticazione Admin

Endpoint a supporto del caso d'uso UC8. L'accesso usa una sola password; al successo viene emesso un token di sessione (Bearer) richiesto dalle operazioni riservate. La password è verificata contro l'hash `bcrypt` dell'utente `admin`; non transita né viene memorizzata in chiaro.

Rispetto alle convenzioni REST dell'Iterazione 1 (corpo d'errore `{"errore": "...}"` con codici 400/404/409/500), l'autenticazione estende l'insieme con il codice **401 Unauthorized**, restituito per password errata o token assente/scaduto; il formato del corpo d'errore resta invariato.

POST /api/auth/admin/login

Verifica la password e, se corretta, restituisce un token di sessione con la relativa scadenza (epoch in millisecondi). Campo obbligatorio: `password`. Risponde 401 se la password è errata e 400 se assente.

```
// Richiesta
{ "password": "1111" }

// Risposta 200
{ "token": "9f2c1ab3...e07", "scadenza": 1769616000000, "ruolo": "admin" }
```

GET /api/auth/admin/verifica

Conferma che il token presentato nell'header `Authorization: Bearer <token>` è ancora valido. Usato dal frontend al caricamento della pagina admin per decidere se

mostrare il pannello o la schermata di accesso. Risponde 401 se il token è assente, scaduto o non valido.

```
{ "valido": true, "ruolo": "admin" }
```

POST /api/auth/admin/logout

Revoca il token corrente (operazione protetta). Dopo il logout il token non è più utilizzabile.

```
{ "messaggio": "Logout effettuato" }
```

PUT /api/auth/admin/password

Modifica la password admin (operazione protetta dal token). Verifica la password attuale prima di salvare l'hash della nuova. Campi obbligatori: `password_attuale` e `nuova_password`. Risponde 401 se la password attuale è errata o il token è assente.

```
// Richiesta
{ "password_attuale": "1111", "nuova_password": "2468" }

// Risposta 200
{ "messaggio": "Password aggiornata" }
```

Health

GET /api/health

Endpoint di controllo vita del server. Non richiede autenticazione; può essere interrogato da load balancer o strumenti di monitoring.

```
{ "stato": "ok", "timestamp": "2026-05-26T20:25:00.000Z" }
```